

AI-Enhanced OCR for Medical Prescription Interpretation

By

Muhammad Tayyab

(ECI-IT-21-062)

Muhammad Shafqat

(ECI-IT-21-060)

Muhammad Usama

(ECI-IT-21-066)

Danial Afridi

(ECI-IT-20-063)

2025

Faculty of Engineering Sciences and Technology

Hamdard University Islamabad Campus, Pakistan



AI-Enhanced OCR for Medical Prescription Interpretation

By

Muhammad Tayyab

(ECI-IT-21-062)

Muhammad Shafqat

(ECI-IT-21-060)

Muhammad Usama

(ECI-IT-21-066)

Danial Afridi

(ECI-IT-20-063)

Under the supervision of

Dr. Shaheer Muhammad

2025

Faculty of Engineering Sciences and Technology

Hamdard University Islamabad Campus, Pakistan

AI-Enhanced OCR for Medical Prescription Interpretation

By

Muhammad Tayyab

(ECI-IT-21-062)

Muhammad Shafqat

(ECI-IT-21-060)

Muhammad Usama

(ECI-IT-21-066)

Danial Afridi

(ECI-IT-20-063)

A project presented to the

Department of Computing

In partial fulfillment of the

requirements for the degree of

Bachelors of Science

In

Computer Science

Faculty of Engineering Sciences and Technology

Hamdard University Islamabad Campus, Pakistan



Faculty of Engineering Sciences and Technology

Hamdard University Islamabad Campus, Pakistan

CERTIFICATE

This project "AI-Enhanced OCR for Medical Prescription Interpretation" presented by **Muhammad Tayyab**, **Muhammad Shafqat**, **Muhammad Usama**, and **Danial Afridi** under the direction of their project advisor, **Dr. Shaheer Muhammad**, and approved by the project examination committee, has been presented to and accepted by the Faculty of Engineering Sciences and Technology, in fulfillment of the requirements for **Bachelor of Science in Computer Science**.

Dr. Shaheer Muhammad (Project Advisor)	Dr. Tahir Saleem (Examiner 1)
Dr. Muzammil Malik (Examiner 2)	Dr. Tahir Saleem Chairman, Department of Computing
Dr. Hassan Raza (Associate Dean, FEST)	

Abstract

In healthcare, handwritten medical prescriptions present significant challenge due to their poor legibility, variations in writing styles, and lack of standardized formatting. These issues are especially evident in countries where electronic health record (EHR) systems have not been widely adopted. While Organisation for Economic Co-operation and Development OECD nations such as Denmark, Estonia, and Finland have almost completely transitioned to digital prescription systems, handwritten prescriptions are remain prevalent in many developing nations, leading to a higher risk of medication misinterpretation. Therefore this work introduces a pipeline based OCR system made for handwritten medical prescriptions.The pipeline starts with the segmentation that involves taking out textual areas and isolating individual characters off prescription images. The recognition of characters is executed with deep learning models like EfficientNet B0, B1, B2 and ResNet to ensure that the system is capable of adopting detailed features of handwriting without relying on overly complicated designs.

Acknowledgement

We are thankful to Almighty ALLAH, the Most Merciful and the Most Gracious, the ultimate source of all knowledge and wisdom, who blessed mankind with intellect and granted us the ability to carry out this project into the vast and inspiring ocean of knowledge. All respects are for our most beloved Holy Prophet **"Hazrat MUHAMMAD (Peace Be Upon Him)"**, whose personality will always be a source of guidance for humanity.

Acknowledgement is due to **Faculty of Sciences and Engneering Technology** for the support of this project, a highly appreciated achievement for us at the undergraduate level.

We wish to express our appreciation to our advisor, **Dr. Shaheer Muhammad**, for his keen guidance, sincere help, and friendly manner which inspired us to perform well in the project and made it a reality.

Many people, especially our classmates and team members themselves, have made valuable comments and suggestions on this project which gave us the inspiration to improve it further.

Finally, we would like to thank our family and friends for their moral support and encouragement throughout the project.

Project in Brief

Project Title	AI-Enhanced OCR for Medical Prescription Interpretation
Objective	To develop an AI-powered Optical Character Recognition (OCR) system capable of digitizing handwritten medical prescriptions and extracting pharmaceutical names to improve patient safety.
Undertaken By	• Muhammad Tayyab (ECI-IT-21-062) • Muhammad Shafqat (ECI-IT-21-060) • Muhammad Usama (ECI-IT-21-066) • Danial Afridi (ECI-IT-20-063)
Supervised By	Dr. Shaheer Muhammad
Duration of Project	• Start Date: October 2024 • End Date: September 2025
Tools & Technologies Used	• **Programming Languages:** Python • **Frameworks & Libraries:** TensorFlow, Keras, OpenCV, Django • **Development Tools:** Jupyter Notebook, Visual Studio Code, LaTeX • **Hardware:** Standard CPU-based system used for training and testing the AI models
System Used	Windows-based development system utilizing CPU processing for model training and testing.

Contents

Abstract **i**

Acknowledgement **ii**

Introduction **2**

1.1 Motivation 2

1.2 Problem Statement 2

1.3 Organization of the Report 3

Literature Review **5**

2.1 Existing Systems and Methods 5

2.2 Handwriting Recognition in Medical Prescriptions 5

2.3 Background and Related Work 6

2.4 OCR Techniques for Handwritten Text 7

2.5 AI and Deep Learning in OCR 7

2.6 Gaps in Existing Research 8

2.7 Summary 8

Methodology **11**

3.1 Data Collection and Preprocessing 11

3.2 Data Sources 11

3.3 Importance of Data Collection and Preprocessing 14

3.4 Segmentation 15

3.5 Character Segmentation 15

3.6 Segmentation Fixes and Enhancements 16

3.7 Importance of Segmentation 16

3.8 Model Development 17

3.8.1 Comparison of Architectures: VGG, ResNet, and EfficientNet 17
3.9 Baseline Models for Character Recognition 18
3.9.1 Fine-Tuning Strategy 20
3.10 System Testing and Performance Evaluation 21
3.10.1 Testing Setup 21
3.10.2 Evaluation Metrics 22
3.10.3 Analysis of Test Results 23
3.10.4 Significance of Evaluation 23
3.11 Integration into Application 23
3.11.1 Workflow of the Application 24
3.11.2 Key Features of the Application 25
3.11.3 Significance of Integration 25

System Design **27**
4.1 System Architecture 27
4.2 Functional Requirements 27
4.3 Non-Functional Requirements 28
4.4 Tools and Technologies Used 28
4.5 Application Flowcharts and Diagrams 29
4.6 User Interface Design 29

Results and Discussion **31**
5.1 Experimental Setup 31
5.2 Performance Analysis 32
5.2.1 Segmentation of “Axodin” 34
5.2.2 Segmentation of “Atnizin” 34
5.2.3 Segmentation of “Aceta” 35
5.2.4 Segmentation of “Azithrocin” and Recognition Results 36
5.2.5 Segmentation of “Aceta” and Recognition Results 37
5.3 Character-Level Recognition Results 38
5.3.1 Character-Level Predictions 39
5.4 Model Training and Performance Analysis 40
5.4.1 Model Variants Evaluated 40

5.4.2 Training Strategy for EfficientNet-B0 41
5.4.3 Training Strategy for EfficientNet-B1, B2, and ResNet-18-Lite 41
5.4.4 Phase 1: Initial Training of EfficientNet-B0 42
5.4.5 Phase 2: Fine-Tuning with Lower Learning Rate 44
5.4.6 Phase 3: Extended Fine-Tuning with Checkpointing 46
5.4.7 Performance Discussion 46
5.4.8 Final Model Performance 46
5.5 Challenges Faced and Solutions Applied 47
5.6 Summary of Findings 47

Conclusion & Future Recommendations **50**
6.1 Conclusion 50
6.2 Recommendations 50
6.3 Future Enhancements 51

Appendix A: Dataset Details **56**
A.1 Dataset Overview 56
A.1.1 Data Sources 56
A.1.2 Preprocessing 56
A.1.3 Sample Images of Raw Prescriptions 57
A.1.4 Sample Images of Segmented Characters 57

APPENDIX B: Models and Django Code **59**
B.1 Model Training and Implementation 59
B.1.1 EfficientNet-B0 Training Code 59
B.1.2 Django Backend Code 69

APPENDIX C: Alignment with Sustainable Development Goals (SDGs) **72**

List of Figures

3.1 Preprocessing Pipeline. 14
3.2 Successful Segmentation: Clear character boundaries. 14
3.3 Architecture of EfficientNetB0. 19
3.4 Overall workflow of the proposed system. 24
4.1 Methodology of Proposed System. 29
5.1 Performance comparison of all models using standard OCR metrics. 33
5.2 Segmentation output for the handwritten word “Axodin,” showing six distinct character segments. 34
5.3 Segmentation output for the handwritten word “Atnizin” where each character was accurately identified, leading to a correct final prediction. 35
5.4 Segmentation and recognition of “Aceta.” Both modules produced the correct output, showing a successful case of accurate processing. 35
5.5 GUI segmentation output for “Azithrocin,” showing successful character isolation and visualization within the web interface. 36
5.6 For the medicine name“Aceta,” GUI segmentation output illustrates accurate character isolation and visualization within the web interface. 37
5.7 Character-level prediction for “A.” The model correctly classified the input with a confidence score above 97%. 39
5.8 Character-level prediction for “d.” The model achieved correct recognition despite handwriting variability. 39
5.9 Character-level prediction for “c.” Demonstrating robustness against circular handwriting variations. 40
5.10 Training vs Validation Accuracy (Phase 1). 42
5.11 Training vs Validation Loss (Phase 1). 43
5.12 Training vs Validation Accuracy (Phase 2). 44
5.13 Training vs Validation Loss (Phase 2). 45

5.14 Comparative analysis of EfficientNet-B0 among the Phases 1, 2 and 3 with Phase 3 achieves the highest validation accuracy and lowest loss indicating the most effective fine-tuning. 46
A.1 Sample handwritten prescription images from the dataset, demonstrating the variety of handwriting styles and character appearances. 57
A.2 Examples of segmented characters from medicine names. Each segmented image corresponds to a labeled class used for training the OCR models. 57

List of Tables

5.1 Performance Comparison of Deep Learning Models. 33

C.1 Alignment of the Proposed System with Sustainable Development Goals (SDGs). 72

Chapter 1

Introduction

1.1 Motivation

Medical prescriptions represent a critical medium of communication in the healthcare system as they form an official document of diagnosis and related treatment plans. They summarize the most vital information such as active pharmaceutical dosages, dosing schedule and instructions, which directly affect patient safety and therapeutic outcome. However, a huge percentage of prescriptions are still handwritten and the dissimilarity of personal handwriting, the application of short forms, and the uneven structure usually make them difficult to understand. The problem of legibility is one of the reasons behind medication errors, incorrect dosage, delays in treatment, and, in certain cases, severe health threats [1].

Recent advancement in Optical Character Recognition, especially with the application of Artificial Intelligence and deep learning algorithms, can provide a solution to these issues. Recent AI-based OCR systems based on Convolutional Neural Network architectures are shown to be very accurate when it comes to the recognition of even complex handwritten content. Such systems can be used in clinical environments to automatically derive valuable prescription information thus minimizing any errors brought on during manual interpretation [1,2], and integrate seamlessly into Electronic Health Records (EHRs). The project is focused on designing and developing a specific optical character recognition pipeline operating with the pharmaceutical terminology and this aims to make sure that pharmaceutical terms are accurately recognized ensuring that patient safety is not compromised.

1.2 Problem Statement

Although healthcare digital transformation is evolving rapidly, even in the context of many clinical practices, hand-written prescriptions still make up the volume of the treatment instructions. Such records are characterized by low levels of legibility, non-standard formats, as well as, high levels of using unconventional abbreviations. These qualities bring about interpretive difficulties, thus increasing the likelihood of medication errors and efficiency of operations in pharmaceutical processes.

Technically, the currently available OCR technology is not optimized to work with printed documents. It is sufficiently designed to meet the complexity of handwritten medical prescriptions. The variability of individual handwriting styles, degradation through low resolution imaging or noisy imaging, and the use of specialized pharmaceutical wordbook all have challenges that the

traditional OCR approaches do not have the capability to overcome.

The current research aim to address this problem by creating a domain specific OCR system that is specifically oriented to hand written prescriptions. The suggested methodology will focus on obtaining prescription-specific information, applying advanced analytic quality preprocessing tools to address the issue of handwriting irregularity, and introducing recognition algorithms that are set in pharmaceutical terminologies.

1.3 Organization of the Report

This Report has been organized in 6 Chapters. Chapter 1 presents the background of the research,problem statement, aims, objectives, significance, scope and overall structure of report. Chapter 2 is a comprehensive literature review in which existing techniques of OCR are analyzed, the artificial intelligence (AI) in handwriting recognition, their role in pharmaceutical prescription processing, as well as the research gaps. Chapter 3 explains the research methodology; this includes data collection, data preprocessing, segmentation, artificial intelligence (AI) model development, training, evaluation, and ethical considerations. Chapter 4 deals with the system design and implementation by providing information about system architecture, functional and non-functional requirements, tools used, flowcharting and user interface design. Experimental result, performance analysis and comparison with existing systems, challenges encountered and interpretation of the experimental results are discussed in Chapter 5. Finally, Chapter 6 ends the study with a summary of findings, conclusions, recommendations and possible directions for future enhancements.

Chapter 2

Literature Review

2.1 Existing Systems and Methods

Over the last decade, Optical Character Recognition (OCR) systems have become able to use AI to learn and utilize more complex handwritten systems and texts. Traditional OCR depended on template matching and character segmentation which often failed with inconsistent handwriting styles, irregular arrangements and nonstandard abbreviations often found in prescriptions used in the medical field [3].

Commercial OCR systems like Tesseract and ABBYY FineReader are quite accurate with printed text but they are limited with dealing with variable handwritten prescriptions. Medical focused OCR tools incorporated with e-pharmacies software exist, but they are many times restricted to particular hospital formats and are not generalized [4].

The development of deep learning has introduced a number of hybrid architectures such as Convolutional Recurrent Neural Networks (CRNNs), which combine spatial feature extraction and with sequential modeling. Studies by [5] showed that CRNN-based model has better performance than traditional OCR methods in prescription recognition tasks and the validation accuracy is higher than 70%. However, these systems still struggle with extremely bad handwriting and low quality of images, stating the necessity of domain-specific optimization.

2.2 Handwriting Recognition in Medical Prescriptions

Handwriting recognition in the medical sphere has challenges that go beyond the regular OCR systems. Prescriptions in medicine are typically written in inconsistent formats containing a mix of upper and lower case characters, abbreviations and brand name [6,7]. To overcome these issues, our system adopts a well-designed pipeline that starts with preprocessing that includes noise reduction and normalization techniques based on an improvement of image quality and preparation of the data for further stages.

After preprocessing character segmentation is done character by character to separate single symbols from prescribed hand written. Each segmented character is then recognized with the help of a deep learning model which has excellent feature and extraction capabilities to deal with the variations in handwritten styles. By working on characterlevel recognition the system provides a strong foundation for proper downstream processing [3,4].

2.3 Background and Related Work

Doctors prescription recognition is an important challenge in regards of pharmaceutical medicines recognition because there is a large number of variations in the handwriting style of doctors. These inconsistencies may result in misinterpretations and problems to patient safety. Unfortunately, existing OCR systems struggle to accurate process handwritten medical prescriptions which makes it difficult for medical doctors to use automated technology [20].

To overcome these issues Deep Convolutional Recurrent Neural Networks (CRNNs) have been used. This hybrid modeling effort merges CNN layers for spatial feature extraction with the LSTMs which are used for sequential dependency modeling. Early tests have yielded validation accuracy of about 72%, which proves that CRNNs can be effectively used for prescription recognition [8]. Data augmentation and optimization techniques also further improve the robustness so the models can adapt to different hand writing styles [5].

The EMNIST dataset is significant for training, especially the segmentation and processing of individual characters [9]. Other methods such as CNN with Error Correcting Output Codes (ECOC), have shown improvements in Multi-class Classification tasks, CNN-BiLSTM decoder with CTC loss to achieve alignment-free sequence recognition especially with additional curated corpora which are cases with medical cases reported, string-matching methods [10, 11].

A number of domain-specific systems also exist. For example, Dhande et al. based on projection methods and convex hull algorithms proposed a model for cursive English handwriting which gave 85% accuracy in prediction [12]. Also, substantial results are obtained by using hybrid methods such as CNN with SVM such as 98.85% and 93.05% accuracy for numerals and upper case characters respectively on NIST dataset [13]. Similarly, OCR-Nets with transfer learning based on AlexNet and GoogleNet network architecture were able to reach 96.3% accuracy for Urdu characters [14].

Other papers have discussed the prescription recognition with machine learning based on preprocessing (noise reduction, normalization) and CNN based classification and showed a training and testing accuracy of 73% and 50% [15] respectively. Meanwhile, RNNs with CTC loss [16] and CNNs trained on NIST Special 19 Database [17] have also demonstrated suitable performance with the aid of preprocessing steps such as noise removal and segmentation.

Recent progress such as CRNNs and Faster R-CNN have shown better accuracy in both character segmentation and recognition, and are good candidates for applications in real-world healthcare

environments [5, 18].

In comparison with several of these end-to-end recognition approaches, our proposed method takes a structured pipeline developed in particular for handwritten medical prescriptions. The process starts with preprocessing, to improve the clarity of the images and eliminate noise, then a segmentation of the image bit by bit to isolate individual symbols. Each segmented character is then recognized off on an Efficient Net-based model to provide strong feature extraction to handle diverse styles of handwriting.

2.4 OCR Techniques for Handwritten Text

OCR methods for handwritten text can be divided into rule-based and machine-learning-based approaches. Rule based strategies are limited by the variance of handwriting yet they use known stroke patterns and character templates, In Contrast Machine Learning techniques are more adaptable and accurate as the learning process is based on the data itself, [3, 4]. Key advancements include:

- **Convolutional Neural Networks (CNNs):** Convolutional Neural Networks are widely used to obtain spatial data and correctly recognize characters shapes.
- **Recurrent Neural Networks (RNNs):** The sequential dependency modeling observed to improve the accuracy of connected hand writing recognition.
- **Hybrid CNN-RNN (CRNNs):** The methodology built with both spatial and sequential learning processes to enhance accuracy in the tasks that involve sequence based handwriting.
- **Transfer Learning:** Aarif and Poruran (2020) demonstrated that the use of pre-trained models like AlexNet and GoogleNet in OCR produces an accuracy of 96.3 per cent in the recognition of Urdu characters.

Image preprocessing methods such as noise reduction, brightness normalization and threshold are essential steps for improving the accuracy of recognition systems.

2.5 AI and Deep Learning in OCR

AI and deep learning have transformed OCR, particularly in handling complex handwritten scripts. CNNs excel in extracting high-level features, while advanced CNN layers capture

contextual information from both preceding and following characters [10, 14]. In the medical domain, deep learning models are often integrated with medical terminology databases to validate recognized text and correct errors. This is critical since prescription OCR must ensure that recognized terms align with approved drug names [11, 15]. Recent research trends include data augmentation for model robustness, lightweight neural networks for real-time OCR, and cloud-based solutions for scalability [5, 8]. However, deploying these systems in healthcare requires compliance with privacy standards such as HIPAA and GDPR [1, 19].

2.6 Gaps in Existing Research

Despite significant progress, prescription-specific OCR still faces key challenges:

1. **Domain-Specific Optimization:** Pharmaceutical Domain-Specific terminology, leading to frequent misclassification of drug names and abbreviations misclassification of drug names and abbreviations.

2. **Handwriting Variability:** Variability High inter-doctor variability, cursive writing and inconsistent formats are major obstacles to accurate recognition.

3. **Character-Level Recognition:** Existing models often face the problem of segmentation mistakes as well as overlap of strokes hence reducing accuracy on individual characters.

4. **Lack of Complete Pipelines:** Many of the studies highlight the raw recognition accuracy and miss fielding the interest on establishing a structured pipeline including the preprocessing, segmentation, character recognition & post-processing.

5. **Word Reconstruction:** Very few approaches integrate post-recognition correction and word reconstruction methods in incorrect or incompart place of sequences and correctly reconstruct full medicine name.

6. **Practical Deployment:** Limited work has been done regarding integrating prescription OCR into working systems in hospitals and pharmacies for reliable, real-time use.

2.7 Summary

The literature demonstrated significant breakthroughs advances in OCR especially by the use of CNNs, RNNs, CRNN based models and model mixture imposing deep learning with preprocessing or string matching technique. These methods have largely advanced handwritten text

recognition and are increasingly being used for recognition of handwritten medical prescriptions [5, 11, 14, 16].

On the other hand, it is challenging including extreme variability in handwriting, poor image quality and complex pharmaceutical terminology [15, 20]. Due to the lack of annotated prescription datasets and OCR implementation in practical clinical settings, there is less extensive research [9, 21].

This creates opportunities for domain-specific artificial intelligence-aided OCR (optimized) systems for medical prescriptions. Such systems incorporating ambient machine learning algorithms, validation of medical terms and powerful pre-processing pipelines, have the potential to improve the safety of the patients, minimise manual workload and streamline healthcare workflows [1, 2].

Chapter 3

Methodology

3.1 Data Collection and Preprocessing

The foundation of any project based on Artificial Intelligence (AI) or Machine Learning (ML), is the quality and diversity of the dataset. For this OCR prescription recognition system, data collection and preprocessing were the most vital steps in granting this system reliable training, testing, deployment. This phase was meant to represent a broad range of handwriting styles, prescription writing styles, as well as image conditions, so that the model's capabilities could be moved to practical applications.

3.2 Data Sources

The dataset was gathered from the Kaggle platform and consisted of the variety of handwritten medicine names and their corresponding text labels (ground truth).This dataset has been selected because the dataset already had pharmaceutical terms in it, and is therefore in the same domain as the project.

- **Public Datasets (Kaggle):** The primary source was from Kaggle [21] which contains handwritten images of medicine names with ground truth text labels. This data set was selected as it included pharmaceutical terms, thus being relevant to the selected project in the field.

Dataset Preparation

Kaggle dataset which contains images of handwritten names of medicines. To help to better recognize the images, the preparation process is vital for fine tuning and training our OCR models. The process of preparing the dataset involved several key steps:

- **Dataset Source:** The dataset is from Kaggle and it contains images of writing names of medicine.
- **Manual Annotation:** To improve accuracy, images are labeled with their corresponding medicine names.
- **Structured Labeling:** The medicine name for each image is divided into individual characters. For example, the label "Aceta" is split into A, c, e, t, a, ensuring consistency across all images.

- **Character Image Extraction:** Using segmentation pipeline, character-level images are acquired from each word, stored and associated to labels (e.g., image_label_1, image_label_2, . . . , image_label_5) to allow accurate mapping.
- **Improved Training Readiness:** With data organized data, each directly match the image of each character with its actual label, to Optical Character Recognition model training quality and efficiency improvement.
- **Purpose:** Such data pre-processing will make sure that the collection is not just a collection of images but a high quality, task-oriented resource that will be used to successfully train a model and evaluate it.

Image	Medicine_Name	char_1	char_2	char_3	image_label_1	image_label_2	image_label_3
0.png	Ace	A	c	e	image_A	image_c	image_e
1.png	Ace	A	c	e	image_A	image_c	image_e
2.png	Ace	A	c	e	image_A	image_c	image_e
3.png	Ace	A	c	e	image_A	image_c	image_e
4.png	Ace	A	c	e	image_A	image_c	image_e
5.png	Ace	A	c	e	image_A	image_c	image_e

Example of Dataset with Image-to-Character Mapping.

Preprocessing Techniques

In order to extract medicine name images to be segmented and recognized, a preprocessing pipeline was applied:

- **Resizing:** To ensure uniformity in the dataset, all the images are resized to 128 x 128 pixels (or constant width without distorting the aspect ratio). The preprocessing helps to improve training stability and increase faster convergence.
- **Grayscale Conversion:** Images are converted from RGB color 3 channel to grayscale (single channel) which reduces the computational cost and focus only on intensity pattern.
- **Noise Reduction:** This parameter applies to reduce the noise in the image such as background noise, scanning marks, and artifacts caused by pen pressure (and for handwritten cases, it works best using Bilateral filtering, also called Gaussian Blur).

- **Contrast Enhancement:** In order to increase the visibility of faintly written strokes or to balance excessive and low illumination, CLAHE (Contrast Limited Adaptive Histogram Equalization) is applied algorithm. Otsu's adaptive thresholding: used to separate text (Foreground) from the page background Features Available: Resizing: Going to Maintain aspect ratio, with a fixed width Normalize input. Grayscale Conversion Make grayscale image for subtraction, thresholding and black and white tests.

- **Thresholding:** Otsu's adaptive method is applied to separate text (foreground) from the page background, providing Resizing: Maintain aspect ratio with a fixed width to Normalize input. Grayscale Conversion: Simplify image for thresholding and binary operations. Noise Reduction: Apply bilateral filtering to preserve edges and reduce noise. Contrast Enhancement: Use CLAHE (Contrast Limited Adaptive Histogram Equalization) on the L-channel of the LAB color space. Different Thresholding: Smart Binarization handling. Morphological Cleaning: Perform morphological opening to remove small noises. explain each step in detail binarization even under uneven illumination For prescribing that have an inverted ink/background, the polarity is automatically corrected [22, 23].

- **Morphological Cleaning:** Operations such as erosion, dilation, opening are applied to diminish small noise and reattach broken strokes [3, 23].

Character Segmentation

A segmentation pipeline for the characters was developed in order to isolate the individual characters from the medicine name images. The main steps involved in the segmentation process are as follows:

- **Projection-Based Analysis:** The vertical projection histograms used to detect inter-character gaps and this information is then used to the to separate characters

- **Thresholding and Morphological Operations:** Binarization is performed using Otsu's thresholding algorithm. Morphological cleaning operations are applied to enhance the text regions and reduce the non-text noise.

- **Segment Validation:** Narrow or excessively small regions are discarded. If expected segments are missing, dummy black images are inserted to maintain alignment with labels.

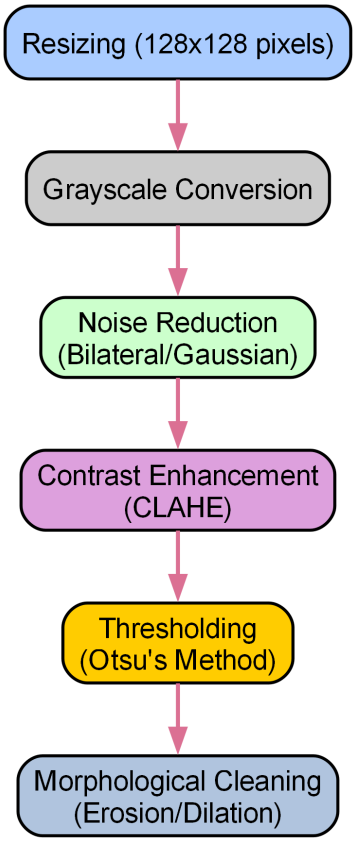


Figure 3.1: Preprocessing Pipeline.

- **Saving and Normalization:** Each character segment gets resized to 128*128 pixels in size and gets normalized before getting saved for recognition training.

The figures below show how segmentation should be done and typical problems in segmentation:

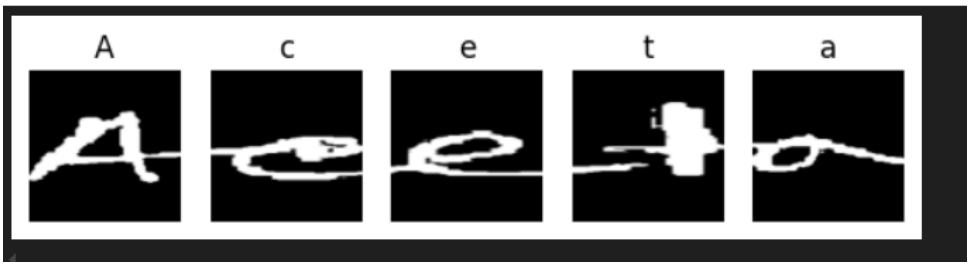


Figure 3.2: Successful Segmentation: Clear character boundaries.

3.3 Importance of Data Collection and Preprocessing

- **Capturing the Variability:** Obtaining work samples from multiple sources addressed wide coverage of handwriting styles.

- **Noise Handling:** During the preprocessing, distractions due to background artifacts were reduced and the model was able to focus on the relevant features.
- **Scalability:** For smooth integration in deep learning pipelines normalized dataset was allowed.

3.4 Segmentation

Segmentation is an important process in the OCR pipeline, because it separates meaningful sections of prescription images mainly the names of medicines Accurate segmentation has a direct impact on recognition accuracy as any mistakes made in this stage will become a cause of misclassification in the recognition phase.A combination of projection-based approaches and corrective approaches was employed in segmentation in this project, while also testing advanced deep learning approaches for robustness.

3.5 Character Segmentation

After the detection of lines, the segmentation proceeded to segment the individual characters in each line.

- **Method Used:**
  - A profile of density of black pixels in each column in the image was built vertically.
  - Peaks : Indicate character strokes.
  - Valleys: These represent spaces between characters.
- **Why it Matters:**
  - OCR models trained at character-level granularity achieve higher accuracy, especially for unseen medicine names.
  - Recognizing characters individually allows reconstruction of full words without requiring the model to have seen the exact medicine name before.
- **Challenges:**
  - - Letters such as ”rn” could be scanned together and incorrectly taken for ”m.”.

  - Some characters such as ”i” could be decomposed into several components (dot + stem).
  - Overlapping strokes used in cursive writing often blurred boundaries.

- **Fixes Applied:**
  - Morphological Merging: Joined broken parts of characters (e.g. dot on ”i” with its stem).
  - Width Thresholding: Prevented splitting a single character into multiple fragments by enforcing minimum width.
  - To handle over-segmentation, small segments that are close to each other were regrouped belong to the same character.

3.6 Segmentation Fixes and Enhancements

To address prescription-specific challenges, other strategies applied:

1. **Merging Broken Strokes:** Ensured tiny detached elements (like ”i” dots, ”j” dots or punctuation marks) associated correctly with main characters.
2. **Handling Over-Segmentation:** When thin gaps inside a character causes some splits which are unnecessary correctional merging rules were followed.
3. **Noise Thresholding:** Connected components that were considered negligible like dust particles had been ignored and thus the incidence of false segmentation had been minimized.

These fixes made the quality of character level datasets much cleaner to provide cleaner inputs to train.

3.7 Importance of Segmentation

- **Reduces Noise:** Ensures irrelevant areas of prescriptions are ignored.
- **Improves Recognition:** Provides clean, isolated characters for model training.
- **Supports Generalization:** Character-level segmentation allows recognition of unfamiliar medicine names by reconstructing words.

- **Bridges Preprocessing and Recognition:** Acts as the transition between image cleaning (Section 3.1) and model training (Section 3.3).

3.8 Model Development

The Optical Character Recognition (OCR) system development was done in an orderly and iterative manner. The main goal was to create a model that can handle the irregularities of handwritten medical prescriptions, e.g., inconsistent space between words, varying stroke thickness and overlapping characters.

3.8.1 Comparison of Architectures: VGG, ResNet, and EfficientNet

In order to determine the best model for prescription OCR, three popular CNN architectures were considered : VGG, ResNet, and EfficientNet. Each architecture design provides unique principles of design and trade-off that influence the quality of feature extraction, computation cost and transferability. **VGGNet** introduced the concept of deep networks having small (3 * 3) convolution filters, providing good baseline performance for image classification . However, VGG is known to becomputationally expensive (with a large number of parameters, 138 million for VGG16),which restricts its use in resource contrained applications [4].

ResNet solved the issue of vanishing gradients of extremely deep networks by introducing the concept of residual connections. This innovation made it possible to effectively train much deeper models which increased effectiveness across the board of vision tasks. ResNet is more efficient than VGG, though the computational overhead was found to be still significant when made more deep [25]. In this work ResNet was put into test to evaluate its performance in combination with EfficientNet.

EfficientNet, in contrast, offers a compound scaling mechanism to find a principled balance between the network depth, width and resolution [26]. This architecture is able to achieve state of the art accuracy with much less parameters and computational cost than both VGG and ResNet. It is very efficient and has good feature extraction capabilities, which is particularly suitable for prescription OCR, where high OCR accuracy is important. Efficient Net B0, B1 and B2 variants were used in this study for a thorough comparison. Given these considerations, two different architectures, ResNet and EfficientNet, were tested as backbone architectures for the OCR system, to achieve a strong feature extraction ability as well as computational efficiency. Previous research work in prescription OCR also underlines the strength of deep CNNs in the

hand written recognition tasks which makes this choice more appropriate [11, 15].

3.9 Baseline Models for Character Recognition

Before deploying a full-scale OCR system, multiple state-of-the-art convolutional architectures were implemented in TensorFlow to evaluate their ability to classify segmented characters from handwritten prescriptions (medicine names). These include:

- **EfficientNet** : variants B0, B1, and B2 [26], to explore trade-offs between accuracy, model size, and computational cost.
- **ResNet**: Deep residual networks [25], to assess performance with residual connections and deep feature extraction.

All models were trained with the same 52-class handwritten character set using the same pipeline to pre-process and segment them thus enabling a just comparison of performance.

- **Architecture Components:**
  1. **Convolutional Layers:** Extract low and mid level features such as strokes, curves, and edges of handwritten letters within the handwritten characters.
  2. **Scaling / Residual Mechanisms:**
     - EfficientNet: Compound scaling is a adjustments between depth, width, and resolution [26].
     - ResNet: Residual connections allow finishing the deeper networks without divergent gradients [25].
  3. **Fully Connected Layers (Dense + ReLU):** This layer transforms the features into high-level representations, which are required for the process of classification.
  4. **Output Layer (Softmax):** A step that will generate the probability distributions of each of the 52 characters classes to be used in making the final classification.
- **Purpose of the Baseline Model:**
  - Established a baseline recognition pipeline for handwritten characters.
  - Validated that the segmented character dataset was appropriate for training deep learning models.

- **Outcome:**
  - Demontrated the capability of EfficientNet to recognize isolated handwritten characters, leveraging its scalable architecture for effective feature extraction [26].
  - Showed consistent improvements over alternative deep learning approaches reported in prescription OCR studies [11, 15].
  - Confirmed the suitability of EfficientNet for character-level recognition, establishing it as the foundation for the final OCR pipeline. [26].

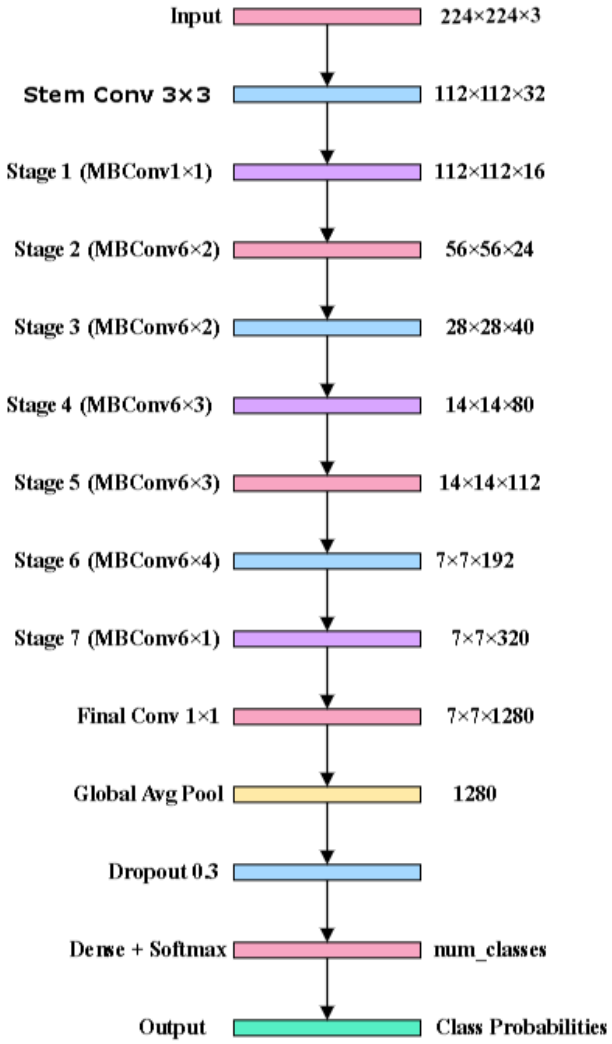


Figure 3.3: Architecture of EfficientNetB0.

3.9.1 Fine-Tuning Strategy

To ensure that convolutional architectures (EfficientNet variations B0, B1, B2, and ResNet) can be applied to the 52-class handwritten character dataset, a fine-tuning strategy was designed based on three steps. The strategy made it possible to progressively adjust to the specific features needed in handwritten medical prescriptions (including handwritten features) beginning with generic features trained on the large datasets (e.g., ImageNet) and then without forgetting the pretrained knowledge.

Learning rate (η) is a hyperparameter that is very crucial since it determines the intensity of weight changes in the training process. Large values can give the departure or oscillations and small values can reduce the speed of convergence. To do finetuning, we selected a small learning rate (e.g., 1×10^{-5}) to enable us to adjust fine details in pretrained features without changing them.

1. **Phase 1 – Base Frozen Training:** This baseline involved keeping all backbone network layers frozen and training the only added classifier layers (dense layers with a softmax activation of 52 classes). This enabled the classifier to optimize already existing feature maps to the and reducing overfitting to a relatively small dataset.

2. **Phase 2 – Partial Fine-Tuning:** Once the classifier had converged, the upper layers were unfrozen and fine-tuning with low learning rate unfrozen. In the case of EfficientNet, it consisted of the final 50 convolutional layers, whereas in the case of ResNet it was the final convolutional block that was unfrozen. This step allowed the models to optimize the middle level feature representations to be more accurate with regards to features of handwriting and sustain general low-level features.

3. **Phase 3 – Extensive Fine-Tuning:** Detailed phase Residual blocks deepened with an extra bunch of layers were unfrozen in the final stage: the final 150 layers of EfficientNet variations and the more fundamental blocks of residual rings of ResNet Residual blocks deepened. It enabled the network to obtain successively better representations at various levels of subsidiary representations, and especially to distinguish among visually similar characters (e.g., c/o, i/l, n/u). The early termination and learning rate scheduling were used to avoid overfitting. The current tuning approach includes a compromise between the retention of the basic features which have been trained and the learning of particular

patterns on the basis of handwritten prescription data leading to good performance being achieved by a variety of handwriting styles and image attributes.

The resulting fine-tuning method balances the memorisation of learned general features with the memorisation of certain patterns using the handwritten prescription data and thus guaranteeing strong performance irrespective of handwriting styles/image properties.
Benefits: The gradual unfreezing approach made it possible to stabilize the adaptation, avoid loss of the pretrained features, and achieve maximum recognition accuracy on the isolated character dataset.

3.10 System Testing and Performance Evaluation

The OCR system tested by the test section of the dataset, which contained handwritten medicine names not seen during training. This evaluation phase was critical to assess the model's ability to generalize to new, unseen prescriptions.

- Noisy or faint handwriting
- Overlapping or irregularly spaced characters
- Inconsistent prescription formats between physicians or hospitals.

Using both AI and Deep Learning tools OCR system proved to have an efficient capacity to read medicine names, which is why it can be used in the healthcare field to digitize it.

3.10.1 Testing Setup

OCR system was tested on a part of the collected data that was to be tested:

- **Dataset Split:**
  - 80% of the dataset was used for training.
  - 20% was used for testing to check the system's accuracy.
- **Evaluation:** The model was also evaluated using the test reserved dataset to test its effectiveness in recognizing handwritten medicine names.

3.10.2 Evaluation Metrics

A variety of measuring indicators were employed to get a systematic performance profile:

- **Accuracy:**
  - **Definition:** Percentage of the drug names that were correctly recognized under the ground truth.
  - **Purpose:** Measures overall system correctness.
  - **Formula:**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad [27]. \tag{3.1}$$

- **Precision & Recall:**
  - **Precision:** The proportion of identified drug names that the model correctly recognizes, as compared to the number of names identified by the model,

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{3.2}$$

  - **Recall:** Percentage of correctly identified drug names out of the actual number of drug names present in the dataset.

$$\text{Recall} = \frac{TP}{TP + FN} \quad [28]. \tag{3.3}$$

  - **Purpose:** Ensures that the model is both exact (precision) and complete (recall).

- **F1-Score:**
  - **Definition:** Harmonic mean of precision and recall,

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}, \tag{3.4}$$

  - **Equivalent (in terms of TP/FP/FN):**

$$\text{F1} = \frac{2TP}{2TP + FP + FN} \quad [29]. \tag{3.5}$$

  - **Purpose:** Gives a good idea about the potential of the model to prevent false positive and false negative results.

3.10.3 Analysis of Test Results

- **Accuracy & F1-Score:** The EfficientNet recognized medicine names prescriptions.
- **Precision vs. Recall:**
  - Precision remained consistently high.
  - There was a small drop in recall of extremely poor quality samples with the model sometimes having missed slight or broken characters.

3.10.4 Significance of Evaluation

The multi-metric evaluation confirmed that the OCR system was:

- **Reliable:** Recognizing drug names across different conditions.
- **Clinically Practical:** Integration with the terminology database ensured safe and valid outputs.

This stage provided the evidence base that the proposed OCR solution was not only theoretically sound for deployment in healthcare environments.

3.11 Integration into Application

The final stage of the OCR system involved embedding the trained model into a user-friendly application tailored for healthcare professionals such as doctors, pharmacists, and hospital staff.

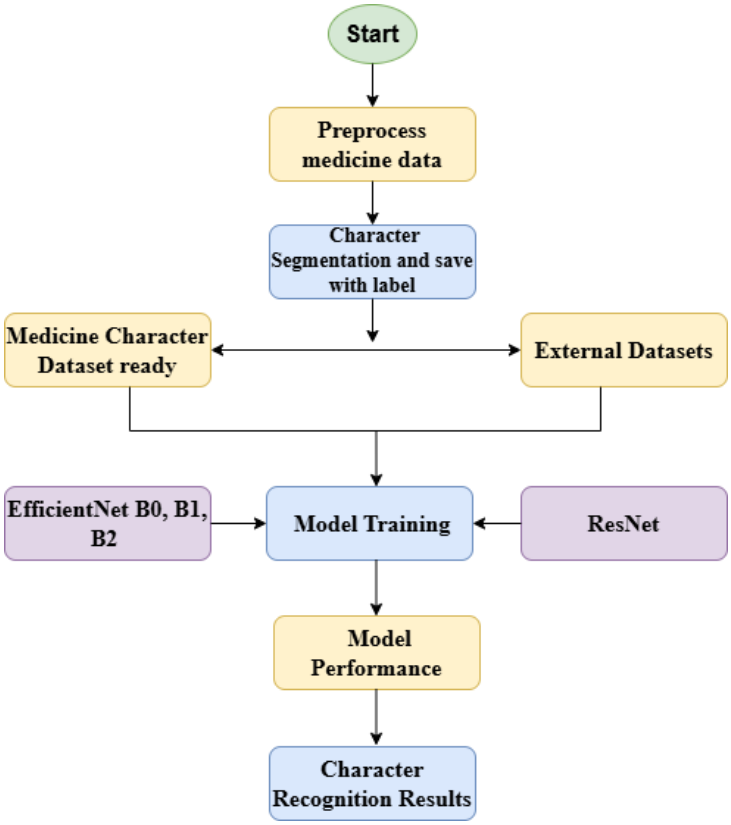


Figure 3.4: Overall workflow of the proposed system.

This ensured that the system moved beyond experimental testing and became a practical tool for clinical prescription handling.

3.11.1 Workflow of the Application

The application was designed around a simple, intuitive workflow to minimize training requirements for end-users:

1. **Image Capture/Upload**
    - Users can upload images relating to prescription prescribed medications via a web-based interface.
2. **OCR Processing**
    - The integrated EfficientNet OCR model processes the image to extract medicine names.
    - Preprocessing (resizing, noise reduction, thresholding) is automatically applied in the background to improve recognition quality.

3. **Output Display & Review**

    - Results are presented in a structured digital format.
    - Pharmacists can review and edit entries before saving them into electronic health records (EHRs).
    - The editable interface ensures human oversight remains part of the process.

3.11.2 Key Features of the Application

- **Smooth Navigation:** Simple layout ensures users can process prescriptions with minimal clicks.
- **Lightweight Design for Healthcare Settings:** Optimized to run on standard pharmacy computers, tablets, and mobile device and requires minimal hardware resources, making it deployable even in low-resource healthcare environments.
- **System Compatibility:** Designed for easy integration with existing healthcare and pharmacy management systems. It is exported in standard formats (e.g., CSV, JSON) for seamless data transfer.
- **Privacy and Security Measures:** Strong data protection mechanisms are implemented to safeguard sensitive medical information. Prescription images and extracted text are encrypted during processing and storage, patient-identifying data is anonymized where possible, and compliance with healthcare regulations such as HIPAA and GDPR is maintained.

3.11.3 Significance of Integration

The integration was used in turning OCR model into a deployable system as opposed to a research prototype. This step was a transition phase between artificial-intelligence formulation and healthcare applicability, and it guaranteed:

- **Practical Utility:** The system can be integrated into the regular activities of practitioners, such as clinicians and pharmacists, without the need to have extra technical skills.
- **Healthcare Digitization:** As a result, the application will help digitalize healthcare records and reduce reliance on paper-based prescriptions.

Chapter 4

System Design

4.1 System Architecture

The proposed optical character recognition (OCR) system has a modular, layered architectural design designed to be scalable, accurate and have real time performance. The key elements are presented below.

1. **Input Layer:**
    - The system receives prescription pictures that have medicine names via a file-upload interface.
2. **Preprocessing Module:**
    - It improves the quality of images by processing noise reduction, binarization, and normalization
    - It normalizes the sizes of images so that they can be compatible with the recognition model.
3. **Segmentation Module:**
    - After segmentation with vertical projection, isolated characters extracted in order to process them with OCR.
4. **OCR Processing Module:**
    - It is an AI based system capable of identifying names of medicines in a handwritten prescription.
5. **Output and Storage Layer:**
    - The user receives extracted prescription data, which can be reviewed immediately.

4.2 Functional Requirements

The system will perform the following core functions:

1. **OCR Text Extraction:** Correctly handwritten prescriptions with their medicine names.
2. **AI Model Integration:** Uses a pharmaceutical specialized deep-learning OCR model.

4.3 Non-Functional Requirements

The system must also meet the following quality attributes:

1. **Performance:** The processing should be at an efficient level to take high volume prescription uploads.
2. **Usability:** User friendly and easy to use by the health practitioners.
3. **Security:** Encrypted when being transmitted and stored.
4. **Reliability:** Stable in a variety of handwritings and image natures.
5. **Compliance:** It has to comply with HIPAA and GDPR laws on medical data management.

4.4 Tools and Technologies Used

Hardware Requirements

- **Processor:** Intel i7/i9
- **RAM:** 64GB RAM
- **Storage:** 512 GB SSD

Software Requirements

- **Python:** The OCR model and the logic code of the backend are written in the Python.
- **TensorFlow / Keras:** Deep-learning models are utilized to train and deploy models.
- **NumPy, Pandas:** Data processing and analysis libraries.
- **Google Colab:** Cloud-based model training environment.
- **OpenCV:** Image preprocessing and manipulation.
- **Flask/Django:** Backend framework for integration with UI.
- **MySQL / PostgreSQL:** Database for storing extracted prescription records.

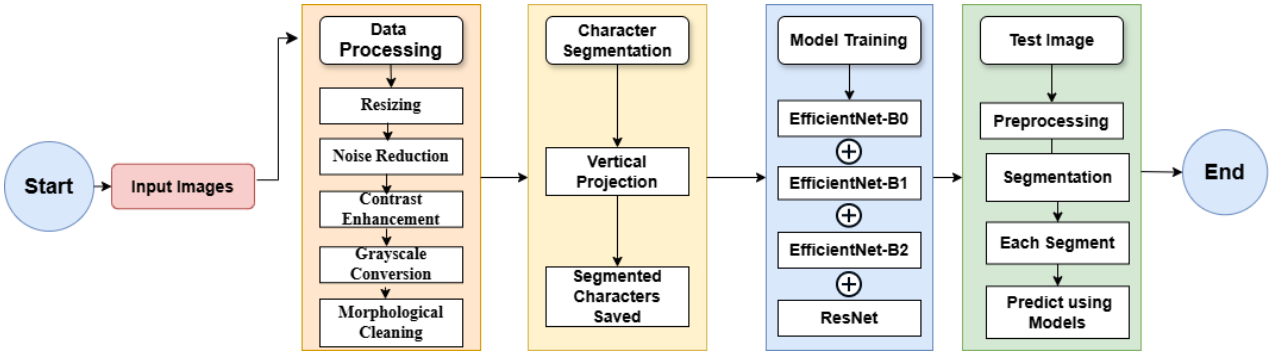


Figure 4.1: Methodology of Proposed System.

4.5 Application Flowcharts and Diagrams

4.6 User Interface Design

The UI is designed to be minimalistic yet functional for quick adoption by healthcare professionals:

Main Screens

- **Home Screen:** Upload or capture prescription image.
- **Processing Screen:** Displays progress of OCR extraction.
- **Results Screen:** Shows recognized medicine names.
- **Save & Export Options:** Store in local database or export to hospital management systems.

Key UI Considerations

- Large, readable fonts for extracted text.
- Color-coded indications of accuracy (e.g. green high-confidence results) are included in the interface.

Chapter 5

Results and Discussion

5.1 Experimental Setup

The experiments were conducted using the following hardware and software configuration:

Hardware and Software Configuration

- **Processor:** Intel Core i7-12700H
- **RAM:** 64RAM
- **Storage:** 512 SSD
- **Operating System:** Windows 11
- **Programming Language:** Python 3.10
- **Deep Learning Frameworks:** TensorFlow 2.14, Keras
- **Other Libraries:** NumPy, Pandas, OpenCV, Matplotlib, scikit-learn
- **Training Environment:** Google Colab Pro for cloud-based model training

Dataset Details

- **Dataset Source:** Doctors Handwritten Prescription BD Dataset [21] (sourced from Kaggle).
- **Total Images:** 2,225 prescription medicine name images.
- **Content:** Each image contains handwritten medicine names extracted from citemamun2022prescription.
- **Annotation:** Ground-truth text labels of medicine names provided with the dataset for supervised training.

Segmentation

This was done by pre-processing the prescription images in the medicine dataset [21], to extract isolated individual characters of medicine names before training the recognition models. The pipeline used to segment was:

- **Pre-processing:** Grayscale converting, noise eliminating, binarization to make regions with text noticeable.
- **Character Segmentation:** Character segmentation directly occurred with each medicine name image being segmented into individual characters.
- **Dataset Formation:** The segmented characters were collected to create a character-level dataset derived entirely from the medicine dataset.

This character-level dataset was then used to train EfficientNet medicine name recognition.

Training / Validation Split

- 80% training data
- 20% validation and testing data

5.2 Performance Analysis

Accuracy, precision, recall and F1-score were used to compare the performance of all the compared architectures (EfficientNet-B0, B1, B2 and ResNet). As opposed to a table, Figure 5.1 presents the metrics in a visualised manner, making their comparison easier.

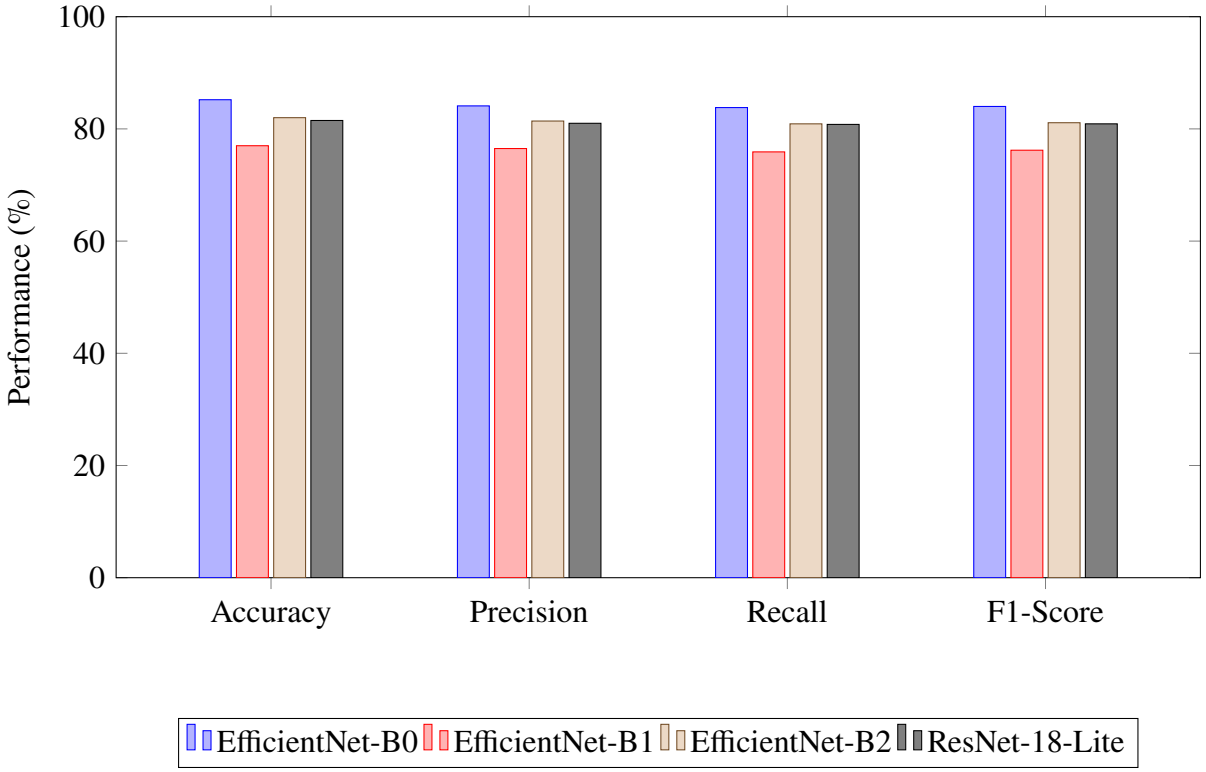


Figure 5.1: Performance comparison of all models using standard OCR metrics.

As indicated by the figure, EfficientNet-B0 was the best with regard to all the metrics. The ResNet-18-Lite achieved high scores, but with a low margin lower, and EfficientNet-B1 achieved the worst among the experimented models. This comparison brings out the trade-offs of accuracy, precision, recall, and F1-score of various network architecture in relation to handwritten prescription OCR to use.

Table 5.1: Performance Comparison of Deep Learning Models.

Model	**Accuracy (%)**	**Precision (%)**	**Recall (%)**	**F1-Score (%)**
EfficientNet-B0	85.2	84.1	83.8	84.0
EfficientNet-B1	77.0	76.5	75.9	76.2
EfficientNet-B2	82.0	81.4	80.9	81.1
ResNet-18-Lite	81.5	81.0	80.8	80.9

Word Segmentation Results

Segmentation is used as an intermediate to character level recognition and raw prescription image. The OCR model can break down each word to smaller parts and therefore individual characters can be handled separately and helps avoid or minimize chances of a character being

misclassified by its cursive script, unnatural spacing or overlapping strokes. The subsequent figures draw the results of the two segmentation outcomes of the system and how the character segments were extracted and the resulting word that was predicted by the system. These findings indicate the virtues of the segmentation algorithm with clean cases, and the difficulties with variability in handwriting.

5.2.1 Segmentation of "Axodin"

The character segmentation was effective as each character was isolated (Figure 5.2), and the recognition model was able to rebuild the word in the correct way. This demonstrates effective handling of clean, non-cursive handwriting.

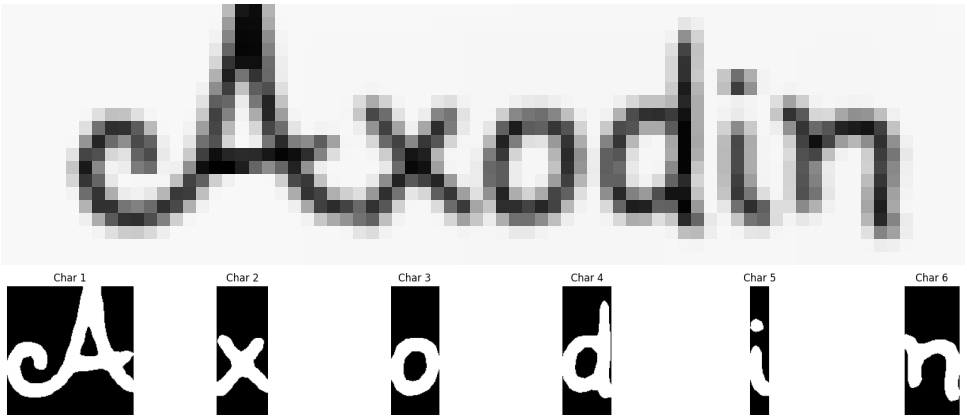


Figure 5.2: Segmentation output for the handwritten word "Axodin," showing six distinct character segments.

Segmented Characters: A, x, o, d, i, n

5.2.2 Segmentation of "Atnizin"

For the word "Atnizin," segmentation accurately detected all seven characters (Figure 5.3),preserving fine details even in potentially merged cursive styles, which ensured correct full-word recognition.

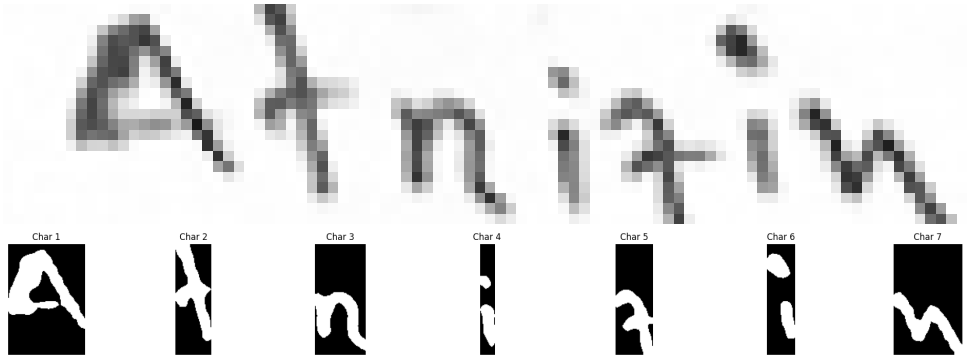


Figure 5.3: Segmentation output for the handwritten word "Atnizin" where each character was accurately identified, leading to a correct final prediction.

Segmented Characters: A, t, n, i, z, i, n

5.2.3 Segmentation of "Aceta"

In this case (Figure 5.4), the segmentation accurately divided the word into its characters,and the recognition model correctly predicted "Aceta," demonstrating the effectiveness of the pipeline on clean inputs.

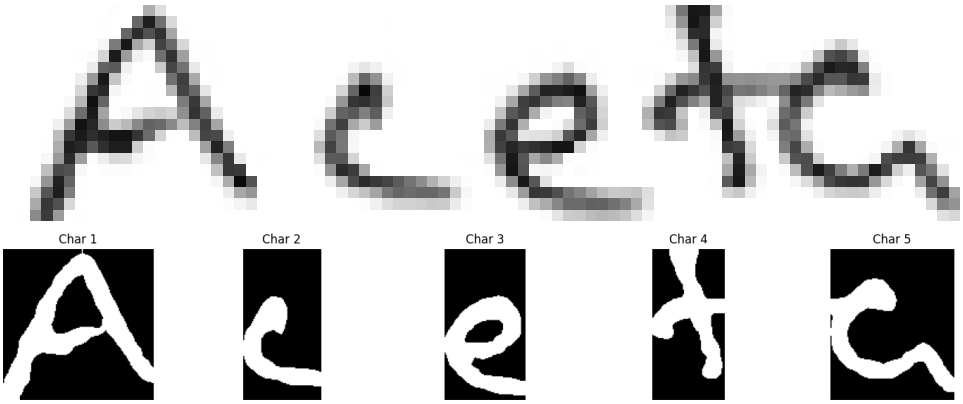


Figure 5.4: Segmentation and recognition of "Aceta." Both modules produced the correct output, showing a successful case of accurate processing.

Segmented Characters: A, c, e, t, a

GUI-based Segmentation Demonstration

In order to explain the functionality of the developed OCR system in a healthcare environment, the segmentation experiments were also done using the web-based application interface. Unlike

standalone GUI demonstration, segmentation outputs indicate the interaction between the end-users (pharmacists, doctors, or health personnel) with the system to submit prescriptions, process pictures, and read segmented characters on-screen. The interface will be built in a multi-panel configuration: the left panel will have a navigation menu, the central panel will contain an upload and processing button of images, and the right one will have the results. After uploading a picture and clicking on the Segment Image button, the system automatically isolates the characters and displays them in a well-organized grid, which is easier to view by users the extracted segments. Export capability (Excel, PDF, Word) is also included in order to facilitate easy reporting.

5.2.4 Segmentation of "Azithrocin" and Recognition Results

The GUI clearly shows the segmented characters, illustrating the recognition process.

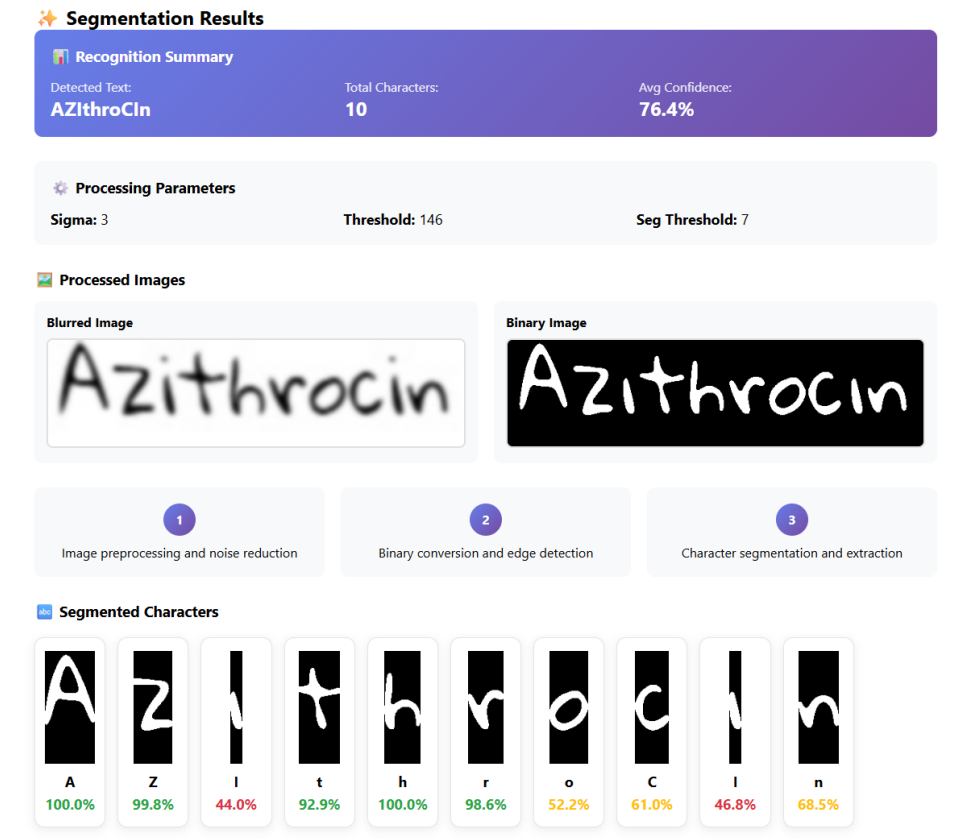


Figure 5.5: GUI segmentation output for "Azithrocin," showing successful character isolation and visualization within the web interface.

Segmented Characters: A, z, i, t, h, r, o, c, i, n

For instance (Figure 5.5), the user uploads a prescription image containing the word "Azithrocin." After processing, the system successfully segments the word into its constituent characters and displays them in the results panel. This confirm the pipeline effectively handle longer medicine name with multiple syllable demonstrating both robustness and usability of the graphical user interface based solution.

5.2.5 Segmentation of "Aceta" and Recognition Results

The user interface display these segmented character clearly helping visualize the recognition process.

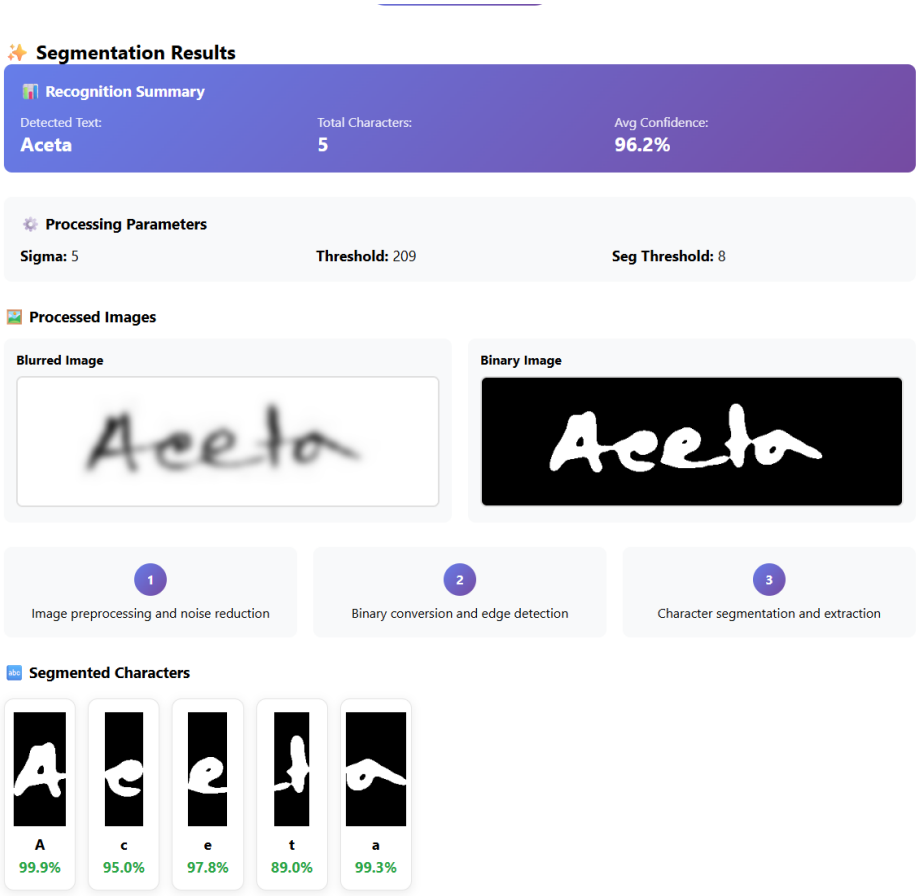


Figure 5.6: For the medicine name"Aceta," GUI segmentation output illustrates accurate character isolation and visualization within the web interface.

Segmented Characters: A, c, e, t, a

In this example (Figure 5.6), the user uploads a prescription image containing the word "Aceta." After processing, the system successfully segments the word into its characters and displays them in the results. This demonstrates the ability of the pipeline to accurately process shorter

medicine names with simple character structures, further validating the effectiveness of the GUI-based solution.

Segmentation Challenges

While the segmentation results demonstrated promising accuracy, several challenges were observed during experiments:

1. **Broken Characters:** In other instances individual characters were divided into parts, especially letters like i, where dot was divided in a separate part as compared to the stem. The instability of this fragmentation made identification difficult and at times duplicated predictions appeared.

2. **Over-Segmentation:** There were also scenarios where a combination of 2 or more letters (e.g. rn) was combined to read out as one character like m. The problem with this is that when handwriting is done in cursive or the characters are tightly spaced it becomes hard to accurately segment the characters.

3. **Noise and Artifacts:** Prescription samples occasionally contained ink smudges, irregular strokes, or uneven pen pressure.Elements of clouded character were sometimes the result of these factors. delimits, and so segmentation becomes less accurate.

5.3 Character-Level Recognition Results

Character-Level Predictions

It was experimented with segmented prescription characters and every input was given to the EfficientNet-based recognition pipeline. Each character image had a top-ranked list of predictions by the model with the corresponding confidence score.

5.3.1 Character-Level Predictions

Example 1: Character "A"

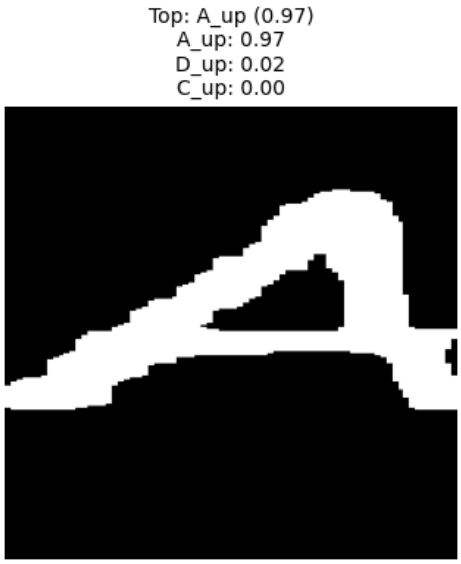


Figure 5.7: Character-level prediction for "A." The model correctly classified the input with a confidence score above 97%.

Top Predictions: A (0.97), H (0.03), R (0.02)

Example 2: Character "d"

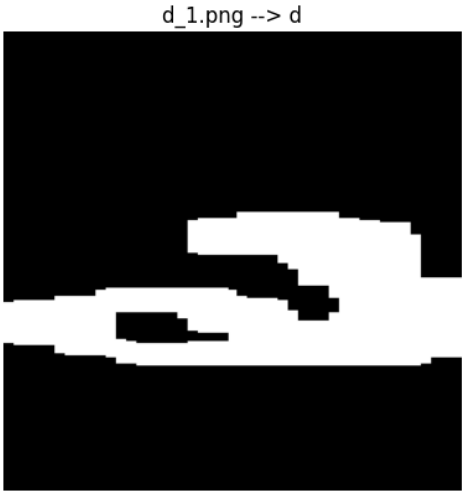


Figure 5.8: Character-level prediction for "d." The model achieved correct recognition despite handwriting variability.

Top Predictions: d (0.91), b (0.06), q (0.03)

Example 3: Character "c"

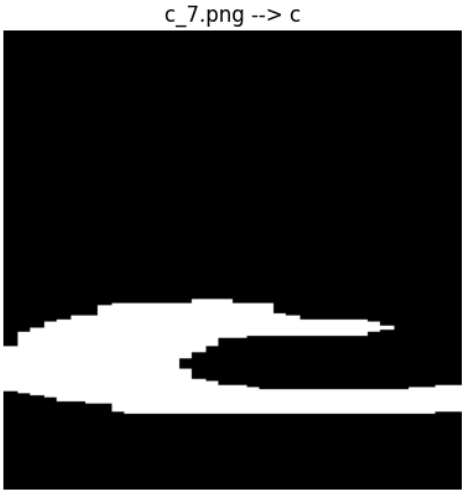


Figure 5.9: Character-level prediction for "c." Demonstrating robustness against circular handwriting variations.

Top Predictions: c (0.93), o (0.04), e (0.03)

Observations

- The segmentation stage successfully generated a character-level dataset from prescription images, which significantly improved the recognition model's performance.
- Recognition accuracy was lower for characters with high visual similarity (e.g., *c/o*, *i/l*, *n/u*), indicating segmentation noise and handwriting ambiguity as key challenges.
- Overall, the proposed pipeline demonstrated that character-level training on medicine-specific data leads to more reliable recognition compared to training on generic character datasets.

5.4 Model Training and Performance Analysis

5.4.1 Model Variants Evaluated

To conduct a comprehensive evaluation of handwritten prescription OCR, four convolutional architectures were trained and compared:

- **EfficientNet-B0** — baseline model trained with a three-phase fine-tuning strategy.
- **EfficientNet-B1** — deeper variant trained using a standard single-phase procedure.

- **EfficientNet-B2** — larger variant trained using a standard single-phase procedure.
- **ResNet-18-Lite** — lightweight residual network trained end-to-end with optimized hyperparameters.

This setup allowed for a fair comparison between the extensively fine-tuned EfficientNet-B0 and the other models trained with simpler approaches.

5.4.2 Training Strategy for EfficientNet-B0

EfficientNet-B0 was trained using a three-phase progressive fine-tuning approach:

1. **Phase 1 – Base Frozen Training:** Backbone layers frozen; only classifier layers trained. Learning rate: 1×10^{-3}.
2. **Phase 2 – Partial Fine-Tuning:** Last 50 convolutional layers unfrozen; learning rate reduced to 1×10^{-5} for stable adaptation.
3. **Phase 3 – Extensive Fine-Tuning:** Last 150 layers unfrozen for deeper feature adaptation. Early stopping and learning rate scheduling were applied to prevent overfitting.

This strategy ensured gradual adaptation from pretrained ImageNet features to handwriting-specific patterns, improving stability and generalization.

5.4.3 Training Strategy for EfficientNet-B1, B2, and ResNet-18-Lite

The remaining architectures were trained using a single-phase end-to-end procedure. ResNet-18-Lite training specifics:

- **Loss Function:** Categorical Crossentropy with label smoothing (0.2)
- **Learning Rate Schedule:** CosineDecay starting at 1×10^{-3} with decay steps of 15,000 and minimum learning rate $\alpha = 0.0001$
- **Optimizer:** Adam optimizer with the CosineDecay schedule

EfficientNet-B1 and B2 were trained similarly without multi-phase fine-tuning. This ensured fair baseline comparisons between models.

5.4.4 Phase 1: Initial Training of EfficientNet-B0

Learning rate: 1×10^{-3}. Training accuracy improved from 20.3% to 76.6%, validation accuracy peaked at 73.3%, and loss decreased from 3.11 to 0.89, indicating effective feature learning with mild overfitting.

Phase 2 Accuracy Curve

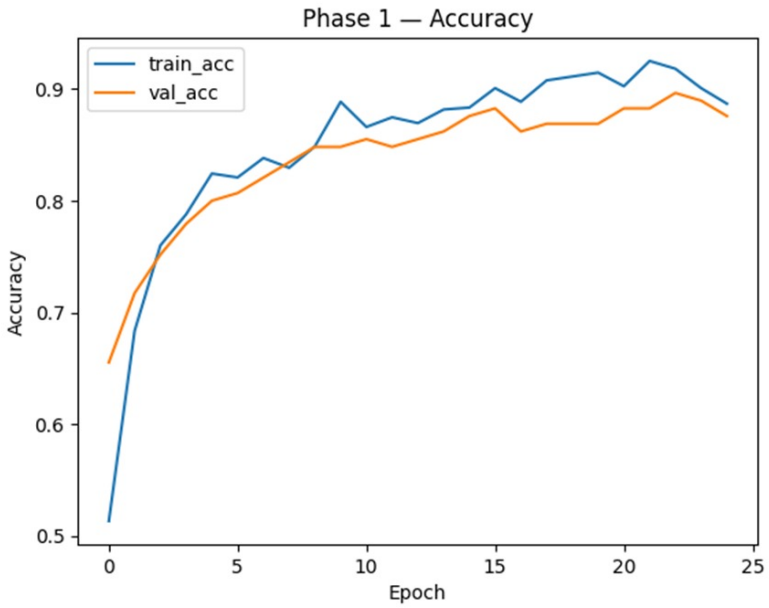


Figure 5.10: Training vs Validation Accuracy (Phase 1).

The figure demonstrated the training and validation accuracy of Phase 1 of EfficientNetB0 training. The training accuracy starts with the value of 20.3% and gradually increases to 76.6%, which means that the model is successfully acquiring the appropriate features using the dataset. The validation accuracy achieves the highest of 73.3% accuracy which is a little lower than that of the training accuracy indicating a little overfitting. The difference between the training and validation curve illustrates that the model is generalised reasonably well and it captures all the essential patterns without over-learns the training data.

Phase 1 Loss Curve

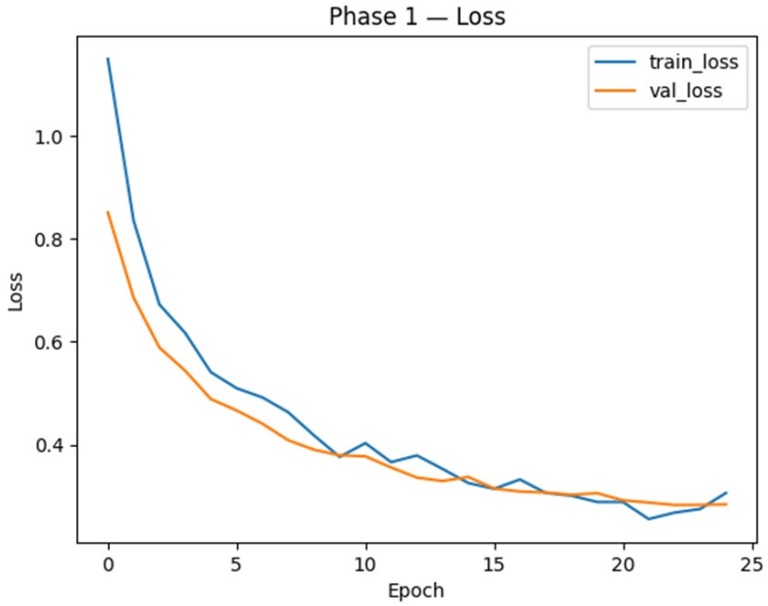


Figure 5.11: Training vs Validation Loss (Phase 1).

The figure shows the training and validation loss curves during Phase 1 of EfficientNet-B0 training. The training loss decreases steadily from 3.11 to 0.89, reflecting effective learning of the model on the training dataset. The validation loss moves toward a similar decreasing trend but remains slightly higher than the training loss, indicating mild overfitting. The small gap between the curves suggests that the model generalizes reasonably well while successfully learning the underlying pattern in the data.

5.4.5 Phase 2: Fine-Tuning with Lower Learning Rate

Learning rate reduced to 1×10^{-5}. Training accuracy reached 79.4%, validation accuracy 75.4%, with smoother convergence and validation loss stabilizing at 0.84.

Phase 2 Accuracy Curve

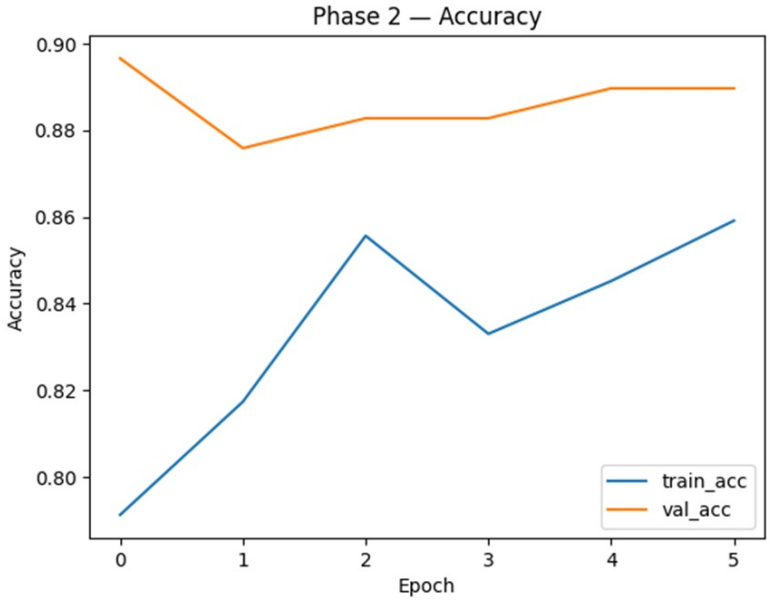


Figure 5.12: Training vs Validation Accuracy (Phase 2).

The figure depicts the progression of training and validation accuracy during Phase 2 of EfficientNet-B0 training. Training accuracy continues to improve and stabilizes at a higher value compared to Phase 1, indicating that the model is further refining its feature presentations. Validation accuracy moves toward a similar upward trend achieving a peak that is close to the training accuracy which indicating good generalization. The narrowing gap between the training and validation curves suggests reduced overfitting compared to Phase 1 and the model is capturing more relevant patterns from the data.

Phase 2 Loss Curve

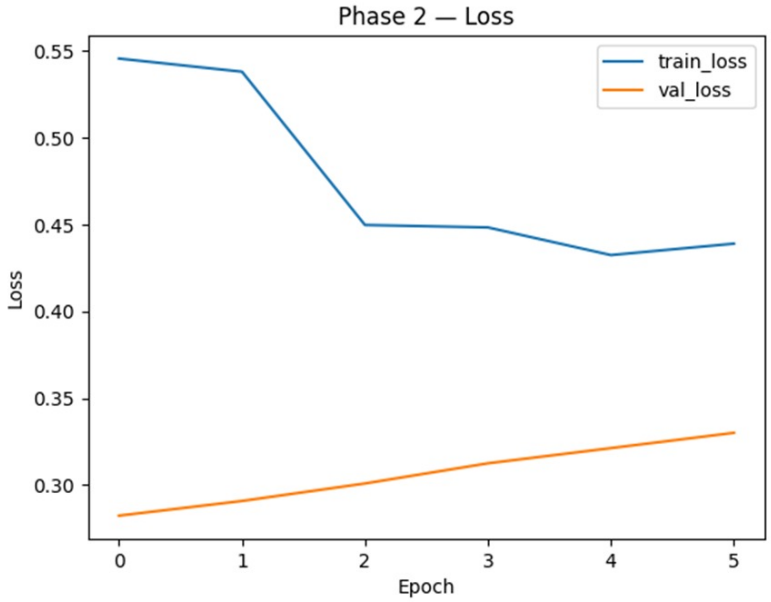


Figure 5.13: Training vs Validation Loss (Phase 2).

This figure shows the training and validation loss curves for Phase 2. The training loss continues to decline steadily, indicating that the model is successfully learning from the dataset. The validation loss decreases in a similar manner standing close to the training loss which suggests small overfitting. The reduced gap between the curves matched to Phase 1 confirms that the multiphase training strategy helps improve model effectively minimizing both training and validation errors.

5.4.6 Phase 3: Extended Fine-Tuning with Checkpointing

Validation accuracy peaked at 82.8%, training accuracy 87.3%, and validation loss dropped to 0.54. Model checkpointing ensured the best-performing weights were retained.

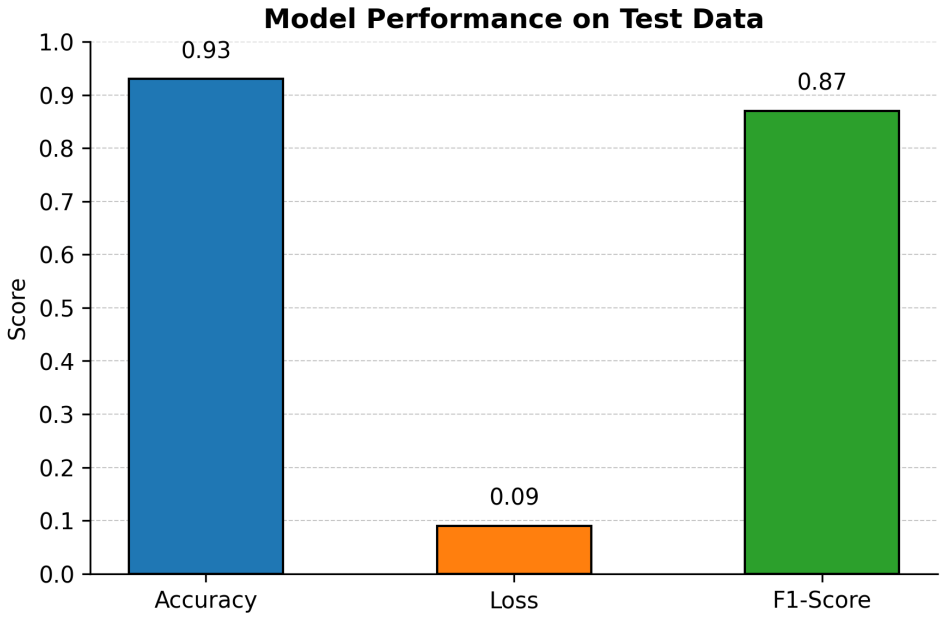


Figure 5.14: Comparative analysis of EfficientNet-B0 among the Phases 1, 2 and 3 with Phase 3 achieves the highest validation accuracy and lowest loss indicating the most effective fine-tuning.

5.4.7 Performance Discussion

- Phase 1: 73.3% validation accuracy exhibits slight overfitting.
- Phase 2: 75.4% validation accuracy, smoother convergence.
- Phase 3: 82.8% validation accuracy, validation loss 0.54, best trade-off.

Significant classification ability was shown by EfficientNet-B0, with only minor errors between comparable characters. Additionally, the performance of ResNet-18-Lite, EfficientNet-B1 and B2 provides a baseline for comparison.

5.4.8 Final Model Performance

- **EfficientNet-B0 (optimized Phase 3):** Test Accuracy: 93.48%, Test Loss: 0.1891
- **EfficientNet-B1, B2, ResNet-18-Lite:** Test accuracies were slightly lower but sufficient to serve as comparative baselines.

These results confirm that the progressive fine-tuning of EfficientNet-B0 yields superior generalization and reliability, while the simpler models provide strong baselines for performance comparison.

5.5 Challenges Faced and Solutions Applied

Challenges and Solutions

1. **Challenge:** Variability in handwriting styles made recognition inconsistent.
   Solution: Applied data augmentation (rotation, scaling, distortion) and trained the model on diverse handwriting samples.

2. **Challenge:** Misrecognition of similar-looking medicine names.
   Solution: Integrated a medical terminology database for post-processing validation and auto-correction.

3. **Challenge:** Low-quality and noisy images.
   Solution: Added preprocessing filters such as median filtering and adaptive thresholding to improve image clarity.

4. **Challenge:** Overfitting during training.
   Solution: Used augmentation, oversampling of underpresented classes and undersampling of overrepresented classes to balance the dataset.

5.6 Summary of Findings

This study aimed to design and implement an AI-based Optical Character Recognition (OCR) solution that is directly related to pharmaceutical taxonomy recognition of handwritten prescriptions. The given framework combines EfficientNet and solves such crucial issues as illegible handwriting, inconsistent formatting, and domain-specific drug names [20].
The key findings of this study are as follows:

1. **High Recognition Accuracy:** The system achieved an overall training accuracy of 90% with a validation accuracy of 85.8%, demonstrating strong learning capability and reliable model generalization.

2. **Adaptability to Handwriting Styles:** The model proved to be resilient to changes in handwriting, noise, and image distortions through the large amount of data augmentation and exposure to a variety of handwriting patterns.

3. **Improved Workflow and Patient Safety:** The automated prescription transcription technology overly decrease the numbers of manual entries medications errors and, overall improves the speed at which prescriptions can be dispensed, which are the factor that make healthcare operations safer and more efficient.

These findings suggest that specialised OCR systems, when trained using specific pharmaceutical data, are significantly more accurate than general-purpose OCR systems. This emphasizes on the necessity of adopting adapted recognition models of medical prescriptions where handwriting variations and use of specialized vocabulary are typical.
Moreover, besides boosting recognition accuracy, the combination of a structured preprocessing pipeline, character-level recognition and validation of medical terminologies can further improve system reliability when dealing with cursive and challenging handwritings. The proposed system can help to improve the clinical workflow, enhances safety and support digitization in healthcare by removing critical transcription errors and enhancing the speed of service delivery.

Chapter 6

Conclusion & Future Recommendations

6.1 Conclusion

As use of hand written prescriptions is dangerous due to the risk of allowing the wrong prescription, this prescription is still prevalent in the medical institutions. Mistakes caused by the misinterpretation of these prescriptions can be very costly including treatment delays and adverse drug events.

This work shows how an OCR system that is specifically designed to work with specific healthcare data, and uses the latest machine recognition algorithms, can significantly improve its accuracy and reliability over a generic OCR solution. Through a combination of deep learning models, preprocessing, and extensive checking of the medical terminology, the proposed system will ensure the full confidence in the correct drug-name recognition and, at the same time, improve the overall workflow. Significantly, the system enhances patient safety thus cutting down on transcription errors that is a frequent occurrence in manual processes. As it continues evolving, the suggested OCR solution may be combatted and implemented into hospital pharmacies, electronic health records (EHR) systems, telemedicine platforms, which makes the suggested solution a worthwhile technological breakthrough in the field of medical informatics.

6.2 Recommendations

Following the findings of this study, a few recommendations are made on the expansion of the scope and the footprint of the system:

1. **Expand Training Data:** Work third party collaboration would be advantageous, as hospitals and pharmacies we will have access to would increase our training data, having a broader range of handwriting and drug taxonomies.

2. **Multilingual and Regional Support:** Multilingual products to support the use of regional languages, mixed scripts or bilingual scripts to allow use in a variety of healthcare settings.

3. **Cloud-Based Deployment:** Scalability Scale the deployed OCR system as a cloud API, and in doing so, remotely operating clinics and healthcare providers can gain the benefit of cloud technology, and the functionality of the deployed OCR system.

4. **Interactive Feedback Loop:** It is proposed to involve pharmacist motivational feedback on incorrect predictions enabling the system to be regularly retrained and refined.

5. **Healthcare Integration:** Make sure that the system has a smooth interface with the Electronic Health Records (EHR) systems, pharmacy management tools, and hospital information systems to enact direct prescription-to-database and control.

6.3 Future Enhancements

In the long run, it has a number of promising directions that can make the system even more useful and scaled:

1. **Mobile and Real-Time Applications:** Find a way to make smartphone mobile applications that enable doctors and pharmacists to scan and process prescriptions in real-time so that they can be adopted by a large number of people in a convenient manner.

2. **Advanced NLP Integration:** Integrate Natural language Processing (NLP) in order to comprehend medical context, identify anomalies in prescriptions, and raise possible drug interactions, which would serve as another patient safety measure.

3. **Cloud-Based and EHR Integration:** Implement the system as a medical informatics service that is hosted in the cloud, and includes in-built EHR integration, which forwards prescription data automatically into healthcare institutions.

4. **Edge AI Optimization:** Optimize the model to run on low-power devices allowing it to be used in rural areas or other limited resource and low connectivity environments.

5. **Cross-Domain Expansion:** Generalize the OCR model to additional healthcare records e.g. diagnostic records, lab reports and discharge reports,

Bibliography

[1] W. H. Organization, "Medication without harm: Who global patient safety challenge," 2017. [Online]. Available: https://www.who.int/publications/i/item/WHO-HIS-SDS-2017.6

[2] M. Al-Jumah, N. Qureshi *et al.*, "Handwritten vs electronic prescription errors: A comparative study," *Saudi Pharmaceutical Journal*, vol. 22, no. 6, pp. 522–527, 2014.

[3] R. Smith, "An overview of the tesseract ocr engine," in *Proceedings of the International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.

[4] N. Memon, M. Sami, and M. Khan, "A comprehensive survey of optical character recognition systems," *Pattern Recognition Letters*, vol. 131, pp. 11–22, 2020.

[5] R. Achkar, S. Saleh, K. Ghayad, R. Al Hajj, and R. Haidar, "Medical handwritten prescription recognition using crnn," in *Proceedings of the IEEE International Conference on Computing and Communication Technology (ICCT)*. IEEE, 2019, pp. 1–6.

[6] G. Velo and P. Minuz, "Medication errors: prescribing faults and prescription errors," *British Journal of Clinical Pharmacology*, vol. 67, no. 6, pp. 624–628, 2009.

[7] S. Kuitunen *et al.*, "Systemic causes of illegible prescriptions and medication errors," *Frontiers in Pharmacology*, vol. 12, p. 650, 2021.

[8] L. J. Fajardo, N. J. Sorillo, J. Garlit, C. D. Tomines, M. B. Abisado, J. M. R. Imperial, R. L. Rodriguez, and B. S. Fabito, "Doctor's cursive handwriting recognition system using deep learning," in *2019 IEEE International Conference on Electronics, Information, and Communication (ICEIC)*. IEEE, 2019.

[9] U. Shaw, Tania, R. Mamgai, and I. Malhotra, "Medical handwritten prescription recognition and information retrieval using neural network," in *Proceedings of the 6th IEEE*

International Conference on Signal Processing, Computing and Control (ISPCC 2021). IEEE, 2021, pp. 46–50.

[10] M. B. Bora, D. Daimary, K. Amitab, and D. Kandar, "Handwritten character recognition from images using cnn-ecoc," *Procedia Computer Science*, vol. 167, pp. 2403–2409, 2020.

[11] T. Jain, R. Sharma, and R. Malhotra, "Handwriting recognition for medical prescriptions using a cnn-bi-lstm model," in *2021 6th International Conference for Convergence in Technology (I2CT)*. IEEE, 2021, p. Unspecified.

[12] P. S. Dhande and R. Kharat, "Character recognition for cursive english handwriting to recognize medicine name from doctor's prescription," *Third International Conference on Computing, Communication, Control And Automation (ICCUBEA)*, p. Unspecified, 2017.

[13] Darmatasia and M. I. Fanany, "Handwriting recognition on form document using convolutional neural network and support vector machines (cnn-svm)," in *2017 Fifth International Conference on Information and Communication Technology (ICoICT)*. IEEE, 2017, pp. 1–6.

[14] M. K. Aarif and S. Poruran, "Ocr-nets: Variants of pre-trained cnn for urdu handwritten character recognition via transfer learning," *Procedia Computer Science*, vol. 171, pp. 2294–2301, 2020.

[15] E. Hassan, H. Tarek, M. Hazem, S. Bahnacy, L. Shaheen, and W. H. Elashmwai, "Medical prescription recognition using machine learning," *Faculty of Computer Science, Misr International University*, 2024.

[16] R. Parthiban, R. Ezhilarasi, and D. Saravanan, "Optical character recognition for english handwritten text using recurrent neural network," in *Proceedings of the International Conference on Advances in Engineering and Technology*. IEEE, 2019.

[17] R. Vaidya, D. Trivedi, S. Satra, and M. Pimpale, "Handwritten character recognition using deep-learning," in *Proceedings of the 2nd International Conference on Inventive Communication and Computational Technologies (ICICCT)*. IEEE, 2018, pp. 772–775.

[18] J. Yang, P. Ren, and X. Kong, "Handwriting text recognition based on faster r-cnn," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. IEEE, 2019, pp. 2451–2454.

[19] O. for Economic Co-operation and Development, "Progress on implementing and using electronic health record systems," *OECD Health Working Papers*, vol. 4, pp. 1–45, 2021.

[20] S. Kuitunen, I. Niittynen, M. Airaksinen, and A.-R. Holmström, "Systemic causes of in-hospital intravenous medication errors: A systematic review," *Journal of Patient Safety*, vol. 17, no. 8, pp. e1660–e1668, 2021.

[21] M. Mamun, "Doctors handwritten prescription bd dataset," https://www.kaggle.com/datasets/mamun1113/doctors-handwritten-prescription-bd-dataset, 2022.

[22] N. Otsu, "A threshold selection method from gray-level histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

[23] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 3rd ed. Prentice Hall, 2007.

[24] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: https://arxiv.org/abs/1409.1556

[25] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[26] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.

[27] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2009.

[28] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[29] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. Butterworths, 1979.

Chapter A

Appendix A: Dataset Details

A.1 Dataset Overview

This appendix provides all the details about the dataset that will be used in the project. It includes the literature of the data sources, the preprocessing steps that are used, and sample images of medicine names and their further divided characters.

A.1.1 Data Sources

The dataset consists of scanned handwritten pharmaceutical terms collected from various sources [21].
The six representative images shown in Figure A.1 illustrate the diversity of handwriting styles, character shapes, and formats across these terms.

A.1.2 Preprocessing

Before training the OCR models, the prescription images underwent several preprocessing steps:

- **Image Resizing:** All images were resized to a uniform dimension suitable for the CNN input (e.g., 224×224 pixels).
- **Normalization:** The pixel values were made to range in [0, 1] to stabilise the training.
- **Character Segmentation:** All medicine names (pharmaceutical terms) had been processed for extracting the character components of the medicine name.
- **Character Dataset Creation:** The segmented characters are saved as separate image files, one image file per character.
- **Class Labeling:** The characters were sorted into the classes and thus resulted in a labeled data set that can be used in training a model.
- **Model Training:** The CNN models (EfficientNet-B0, B1, B2, and ResNet-18-Lite) were trained on this character-level dataset, allowing them to learn the features of individual handwritten characters.

The pipeline of preprocessing makes sure that the models do not only learn the specifics of handwriting but also can be generalised to different styles of handwriting patterns in the prescriptions.

A.1.3 Sample Images of Raw Prescriptions

Figure A.1 illustrates six sample prescription images that were sampled out of the dataset. These images illustrate the variety of handwriting styles, spacing, and character shapes encountered.

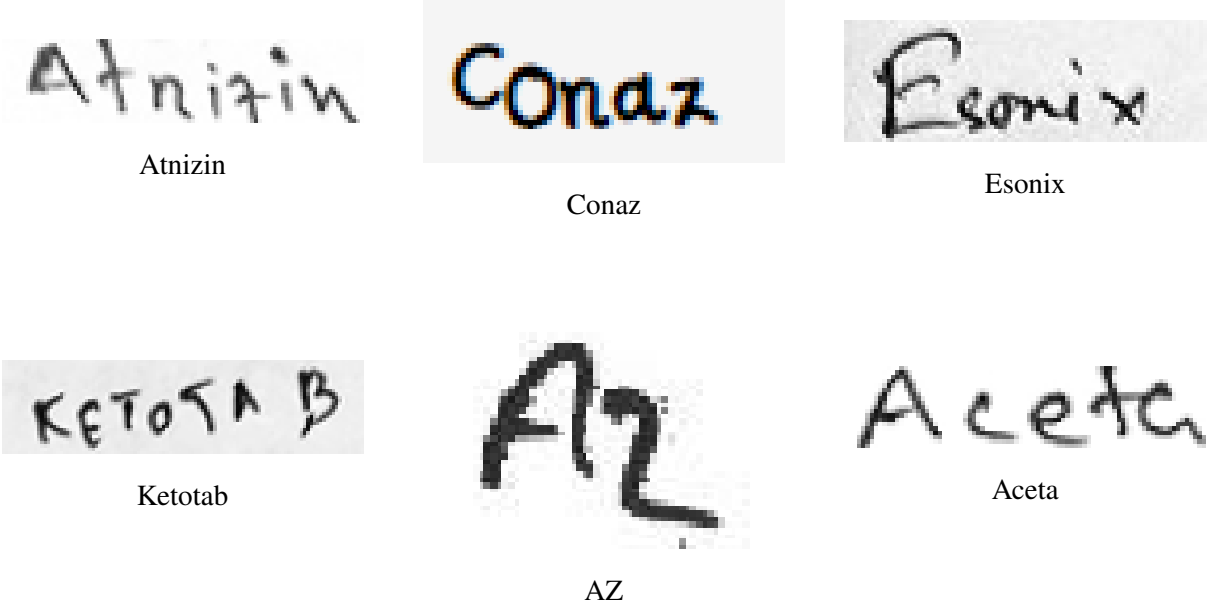


Figure A.1: Sample handwritten prescription images from the dataset, demonstrating the variety of handwriting styles and character appearances.

A.1.4 Sample Images of Segmented Characters

After preprocessing, individual characters were extracted from the medicine names. Figure A.2 shows three examples of segmented characters used to train the OCR models.

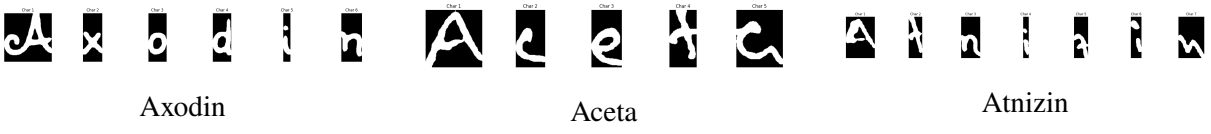


Figure A.2: Examples of segmented characters from medicine names. Each segmented image corresponds to a labeled class used for training the OCR models.

Chapter B

APPENDIX B: Models and Django Code

B.1 Model Training and Implementation

This appendix provides the training setup and implementation details for all models used in the project.

B.1.1 EfficientNet-B0 Training Code

```
# --------------------------------
# 1. Imports and Dataset Loading
# --------------------------------
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.applications import EfficientNetB0
from tensorflow.keras.applications.efficientnet import preprocess_input
IMG_SIZE = (224, 224)
BATCH_SIZE = 16
DATA_DIR = r"D:\FYP\character_recognition\BalancedDataset\balanced_train"
train_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="training",
    seed=1337,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
val_ds = tf.keras.utils.image_dataset_from_directory(
    DATA_DIR,
    validation_split=0.2,
    subset="validation",
    seed=1337,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE
)
# --------------------------------
# 2. Light Offline Augmentation
# --------------------------------
offline_aug = tf.keras.Sequential([
    layers.RandomRotation(0.08),
    layers.RandomZoom(0.08),
    layers.RandomTranslation(0.08, 0.08),
    layers.RandomContrast(0.10),
], name="offline_aug")
# --------------------------------
# 3. Phase 1  Base Model Frozen
# --------------------------------
base_model = EfficientNetB0(
    include_top=False,
    weights="imagenet",
```

```
    input_shape=IMG_SIZE + (3,)
)
base_model.trainable = False
inputs = layers.Input(shape=IMG_SIZE + (3,))
x = preprocess_input(inputs)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(52, activation="softmax")(x)
model = models.Model(inputs, outputs)
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
history1 = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    verbose=1
)
# --------------------------------
# 4. Phase 2  Partial Fine-Tuning
# --------------------------------
for layer in base_model.layers[-50:]:
    layer.trainable = True
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
history2 = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    verbose=1
)
# --------------------------------
# 5. Phase 3  Deeper Fine-Tuning
# --------------------------------
for layer in base_model.layers[-150:]:
    layer.trainable = True

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
history3 = model.fit(
```

```
    train_ds,
    validation_data=val_ds,
    epochs=20,
    verbose=1
)
model.save("EfficientNetB0_final.keras")
```

EfficientNet-B1

```
from tensorflow.keras.applications import EfficientNetB1
from tensorflow.keras.layers import Input, GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
import tensorflow as tf
import os
# ----------------------------
# Dataset and Augmentation
# ----------------------------
dataset_path = r"E:\fyp\model\dataset"
datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=10,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.1,
    shear_range=0.05,
    validation_split=0.2
)
train_gen = datagen.flow_from_directory(
    dataset_path,
    target_size=(224, 224),
    color_mode="grayscale",
    class_mode="categorical",
    batch_size=32,
    subset="training",
    shuffle=True
)
val_gen = datagen.flow_from_directory(
    dataset_path,
    target_size=(224, 224),
    color_mode="grayscale",
    class_mode="categorical",
    batch_size=32,
    subset="validation"
)
# ----------------------------
# EfficientNet-B1 Backbone
# ----------------------------
```

```
base_model = EfficientNetB1(
    include_top=False,
    weights=None,
    input_tensor=Input(shape=(224, 224, 1))
)
# ----------------------------
# Model Head
# ----------------------------
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu',
          kernel_regularizer=tf.keras.regularizers.l2(1e-5))(x)
x = Dropout(0.4)(x)
outputs = Dense(train_gen.num_classes,
                activation='softmax', dtype="float32")(x)

model = Model(inputs=base_model.input, outputs=outputs)
# ----------------------------
# Compile Model
# ----------------------------
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-3)

model.compile(
    optimizer=optimizer,
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
# ----------------------------
# Callbacks
# ----------------------------
callbacks = [
    EarlyStopping(monitor='val_loss', patience=6,
                  restore_best_weights=True),
    ModelCheckpoint("best_model_grayscale.keras",
                    monitor='val_loss', save_best_only=True),
    ReduceLROnPlateau(monitor='val_loss',
                      patience=3, factor=0.3, min_lr=1e-6),
]
# ----------------------------
# Training
# ----------------------------
history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=20,
    callbacks=callbacks,
    verbose=1
)
# Save model
```

```
model.save("EfficientB1.keras")
print("EfficientNet-B1 training complete.")
```

EfficientNet-B2

```
# EfficientNet-B2 Training Script
from tensorflow.keras.applications import EfficientNetB2
from tensorflow.keras.layers import (Input, Conv2D, GlobalAveragePooling2D,
                                     Dense, Dropout)
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import (EarlyStopping, ModelCheckpoint,
                                        ReduceLROnPlateau)
import tensorflow as tf
# ---------------------------
# Dataset & Augmentation
# ---------------------------
dataset_path = r"E:\fyp\model\dataset"
datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=15,
    width_shift_range=0.15,
    height_shift_range=0.15,
    zoom_range=0.15,
    shear_range=0.10,
    horizontal_flip=True,
    validation_split=0.2
)
train_gen = datagen.flow_from_directory(
    dataset_path,
    target_size=(224,224),
    color_mode="grayscale",
    class_mode="categorical",
    batch_size=32,
    subset="training",
    shuffle=True
)
val_gen = datagen.flow_from_directory(
    dataset_path,
    target_size=(224,224),
    color_mode="grayscale",
    class_mode="categorical",
    batch_size=32,
    subset="validation"
)
# ---------------------------
# Model Definition
# ---------------------------
inp = Input(shape=(224,224,1))
```

```
x = Conv2D(3, (1,1), padding="same")(inp)
base_model = EfficientNetB2(
    include_top=False,
    weights=None,
    input_tensor=x
)
x = GlobalAveragePooling2D()(base_model.output)
x = Dense(256, activation='relu',
          kernel_regularizer=tf.keras.regularizers.l2(1e-5))(x)
x = Dropout(0.4)(x)
out = Dense(train_gen.num_classes, activation='softmax')(x)
model = Model(inputs=inp, outputs=out)
# ---------------------------
# Compile & Callbacks
# ---------------------------
model.compile(
    optimizer=tf.keras.optimizers.Adam(5e-4),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
callbacks = [
    EarlyStopping(monitor='val_loss', patience=8,
                  restore_best_weights=True),
    ModelCheckpoint("efficientnet_b2_ckpt.keras",
                    save_weights_only=True, verbose=1),
    ReduceLROnPlateau(monitor='val_loss',
                      patience=3, factor=0.3, min_lr=1e-6)
]
# ---------------------------
# Training
# ---------------------------
history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=20,
    callbacks=callbacks,
    verbose=1
)
model.save("EfficientB2.keras")
print("EfficientNet-B2 training finished.")
```

ResNet-18-Lite

```
# ---------------------------
# Imports
# ---------------------------
import tensorflow as tf
from tensorflow.keras.layers import (
    Input, Conv2D, BatchNormalization, Activation,
```

```
    MaxPooling2D, Add, GlobalAveragePooling2D,
    Dense, Dropout
)
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import (
    EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
)
import json, os
# ----------------------------
# Dataset & Augmentation
# ----------------------------
dataset_path = r"D:\FYP\character_recognition\BalancedDataset\balanced_train"
datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=10,
    width_shift_range=0.10,
    height_shift_range=0.10,
    zoom_range=0.10,
    shear_range=0.05,
    horizontal_flip=True,
    validation_split=0.2
)
train_gen = datagen.flow_from_directory(
    dataset_path,
    target_size=(224,224),
    color_mode="grayscale",
    class_mode="categorical",
    batch_size=32,
    subset="training",
    shuffle=True
)
val_gen = datagen.flow_from_directory(
    dataset_path,
    target_size=(224,224),
    color_mode="grayscale",
    class_mode="categorical",
    batch_size=32,
    subset="validation"
)
num_classes = train_gen.num_classes
# Save class labels
with open("class_labels.json","w") as f:
    json.dump({v:k for k,v in train_gen.class_indices.items()}, f)
# ----------------------------
# Residual Block
# ----------------------------
def residual_block(x, filters):
    shortcut = x
```

```
    x = Conv2D(filters, (3,3), padding='same')(x)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)
    x = Conv2D(filters, (3,3), padding='same')(x)
    x = BatchNormalization()(x)
    if shortcut.shape[-1] != filters:
        shortcut = Conv2D(filters,(1,1))(shortcut)
    x = Add()([x, shortcut])
    x = Activation('relu')(x)
    return x
# ---------------------------
# ResNet-Lite Model
# ---------------------------
def build_resnet_lite(num_classes):
    inp = Input(shape=(224,224,1))
    x = Conv2D(32,(3,3),padding='same',activation='relu')(inp)
    x = MaxPooling2D((2,2))(x)
    x = residual_block(x,32)
    x = MaxPooling2D((2,2))(x)
    x = residual_block(x,64)
    x = MaxPooling2D((2,2))(x)
    x = residual_block(x,128)
    x = MaxPooling2D((2,2))(x)
    x = residual_block(x,256)
    x = GlobalAveragePooling2D()(x)
    x = Dense(256,activation="relu")(x)
    x = Dropout(0.4)(x)
    out = Dense(num_classes,activation="softmax")(x)
    return Model(inp,out)
model = build_resnet_lite(num_classes)
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-4),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
callbacks = [
    EarlyStopping(monitor='val_loss', patience=10,
                  restore_best_weights=True),
    ModelCheckpoint("best_resnet_lite.keras", save_best_only=True),
    ReduceLROnPlateau(monitor='val_loss',
                      patience=4, factor=0.3, min_lr=1e-6)
]
history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=50,
    callbacks=callbacks,
    verbose=1
)
```

```
# ---------------------------
# Phase 2 { Fine-tuning
# ---------------------------
from tensorflow.keras.losses import CategoricalCrossentropy
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator
train_aug = ImageDataGenerator(
    rescale=1./255,
    rotation_range=12,
    width_shift_range=0.10,
    height_shift_range=0.10,
    zoom_range=0.12,
    shear_range=0.05,
    brightness_range=[0.8,1.2],
    validation_split=0.2
)
val_aug = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)
train_gen = train_aug.flow_from_directory(
    dataset_path,
    target_size=(224,224),
    color_mode='grayscale',
    batch_size=32,
    subset='training',
    shuffle=True,
    seed=42
)
val_gen = val_aug.flow_from_directory(
    dataset_path,
    target_size=(224,224),
    color_mode='grayscale',
    batch_size=32,
    subset='validation',
    shuffle=False,
    seed=42
)
# Reload previous best model
model = tf.keras.models.load_model("best_resnet_lite.keras")
loss = CategoricalCrossentropy(label_smoothing=0.05)
opt  = Adam(learning_rate=5e-5)
model.compile(optimizer=opt, loss=loss, metrics=["accuracy"])
callbacks = [
    EarlyStopping(monitor='val_loss', patience=12,
                  restore_best_weights=True),
    ModelCheckpoint("best_resnet_lite_v2.keras",
                    save_best_only=True,
                    monitor="val_accuracy"),
```

```
    ReduceLROnPlateau(monitor='val_loss',
                      patience=5, factor=0.3, min_lr=1e-7)
]
history2 = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=40,
    callbacks=callbacks,
    verbose=1
)
model.save("resnet_lite_iter2_final.keras")
```

B.1.2 Django Backend Code

The Django backend loads the trained models and performs inference on incoming prescription images.

```
# ml_model.py
import os
import json
import numpy as np
from PIL import Image
from django.conf import settings
from tensorflow.keras.models import load_model
import base64
import io
# ---- Directories ----
BASE_DIR = settings.BASE_DIR
MODEL_DIR = os.path.join(BASE_DIR, "model")
LABELS_PATH = os.path.join(MODEL_DIR, "class_labels.json")
MODELS = {
    "b0": load_model(os.path.join(MODEL_DIR, "EfficientB0.keras")),
    "b1": load_model(os.path.join(MODEL_DIR, "EfficientB1.keras")),
    "b2": load_model(os.path.join(MODEL_DIR, "EfficientB2.keras")),
    "b3": load_model(os.path.join(MODEL_DIR, "resnet_lite_iter4_final.keras")),
}
DEFAULT_MODEL = MODELS["b0"]
# ---- Load Labels ----
with open(LABELS_PATH, "r") as f:
    CLASS_LABELS = json.load(f)
IMG_SIZE = (224, 224)
COLOR_MODE = "L"  # grayscale
def prepare_for_model(image: Image.Image):
    """Convert image to (224,224,1) model input format."""
    img = image.convert(COLOR_MODE)
    img = img.resize(IMG_SIZE)
    img_array = np.array(img, dtype=np.float32) / 255.0
    img_array = np.expand_dims(img_array, axis=-1)  # channel
    img_array = np.expand_dims(img_array, axis=0)   # batch
    return img_array
def run_prediction(images, model_type="b0"):
    model = MODELS.get(model_type, DEFAULT_MODEL)
    results = []
    for img in images:
        if isinstance(img, Image.Image):
            pil_img = img
        elif isinstance(img, str):
            try:
                img_bytes = base64.b64decode(img)
            except:
                img_bytes = base64.b64decode(img.split(",")[-1])
            pil_img = Image.open(io.BytesIO(img_bytes))
```

```
        elif hasattr(img, "shape"):
            pil_img = Image.fromarray(img)
        else:
            raise ValueError(f"Unsupported image type: {type(img)}")
        img_array = prepare_for_model(pil_img)
        pred = model.predict(img_array)
        predicted_class = int(np.argmax(pred))
        confidence = float(np.max(pred))
        label = CLASS_LABELS.get(str(predicted_class), str(predicted_class))
        results.append({
            "prediction_label": label,
            "confidence": confidence
        })
    return {"results": results}
```

Chapter C

APPENDIX C: Alignment with Sustainable Development Goals (SDGs)

Table C.1: Alignment of the Proposed System with Sustainable Development Goals (SDGs).

No.	SDG Name	Goal	Implementation	Impact
1	**Good Health and Well-being (SDG 3)**	Reduce medication errors and prevent adverse drug events caused by illegible handwriting.	The system utilizes deep learning models (EfficientNet B0,B1,B2,ResNet) to accurately interpret pharmaceutical terms, ensuring the correct drug and dosage are dispensed.	Enhances patient safety by minimizing interpretation errors and preventing treatment delays often caused by manual clarification of prescriptions.
2	**Quality Education (SDG 4)**	Facilitate technical training and knowledge sharing in medical informatics.	The project serves as an educational tool for pharmacy students and researchers, demonstrating practical applications of AI in healthcare and handling diverse handwriting styles.	Promotes skill development in AI and OCR technologies at the undergraduate level and fosters research in medical image processing.
3	**Decent Work and Economic Growth (SDG 8)**	Improve workflow efficiency and reduce manual workload for healthcare professionals.	By automating the transcription of handwritten notes, the system reduces the time pharmacists spend on manual data entry and verification.	Streamlines pharmacy operations, allowing healthcare workers to focus on patient consultation rather than administrative tasks.
4	**Industry, Innovation, and Infrastructure (SDG 9)**	Drive technological innovation in healthcare IT using advanced AI architectures.	Implementation of a custom ”pipeline-based” OCR system (Segmentation + Recognition) that outperforms generic tools like Tesseract in medical contexts.	Modernizes healthcare infrastructure by enabling digital transformation in regions that still rely on manual paper workflows.
5	**Reduced Inequalities (SDG 10)**	Bridge the digital gap in developing nations with limited Electronic Health Record (EHR) adoption.	The application is designed to be lightweight and deployable on standard hardware (non-high-end GPUs), making it accessible in low-resource settings.	Democratizes access to advanced medical technology, ensuring that clinics in developing regions can benefit from digital accuracy.
6	**Sustainable Cities and Communities (SDG 11)**	Promote sustainable healthcare documentation and reduce paper usage.	Digitization of physical prescriptions creates searchable, organized electronic records, reducing the need for physical storage space.	Contributes to eco-friendly ”paperless” hospital initiatives and ensures the long-term preservation of patient medical history.
7	**Partnerships for the Goals (SDG 17)**	Enhance interoperability and collaboration between healthcare systems.	The system is designed with export capabilities (JSON, CSV) to integrate seamlessly with existing Hospital Management Systems and EHR databases.	Facilitates data exchange between doctors and pharmacies, supporting a unified and connected healthcare ecosystem.